STL Fundamentals

Core Concepts for mastering STL

> Pointers
>
> ## Pointers
>
> Your mental image while dealing with anything related to memory should be that of a box.
>
> Every variable, arrays, everything that you create in a program, will have a name that is local to the program and an address of where it is stored in the RAM.
>
> In C++, a pointer is a variable that stores memory address of another variable. Pointers are one of the most powerful features of C++ because they allow for direct memory access and manipulation, which leads to efficient and flexible code.
>
> ## Key Concepts of Pointers
>
> 1. **Pointer Declaration**
> To declare a pointer, use the `*` symbol.
>
>  ```
>  int* ptr;
>  ```
>
> 2. **Address-of Operator (`&`)**
> The `&` operator is used to get the memory address of a variable.
>
>  ```
>  int x = 10;
>  int* ptr = &x;
>  ```
>
> Here, `ptr` stores the address of variable `x`
> 3. **Dereferencing Operator (`*`)**
> The `*` operator is also used to access the value stored at the memory address a pointer holds (dereferencing the pointer).
>
>  ```
>  int y = *ptr;
>  ```
>
> This assigned the value pointed to by `ptr` (which is the value of `x`) to `y`.
> This is also known as **Value At** operator.
> 4. **Null Pointer**
> A pointer that doesn't point of any memory location is called a null pointer.
> It is usually initialized with `nullptr`
>
>  ```
>  int* ptr = nullptr;
>  ```

5. **Pointer Arithmetic**
   Pointers support arithmetic operations like addition or subtraction, which allows moving the pointer across memory locations, especially in arrays.

   Incrementing or decrementing a pointer moves it to point to the next or previous memory location, respectively. Pointer arithmetic depends on the size of the data type the pointer points to.

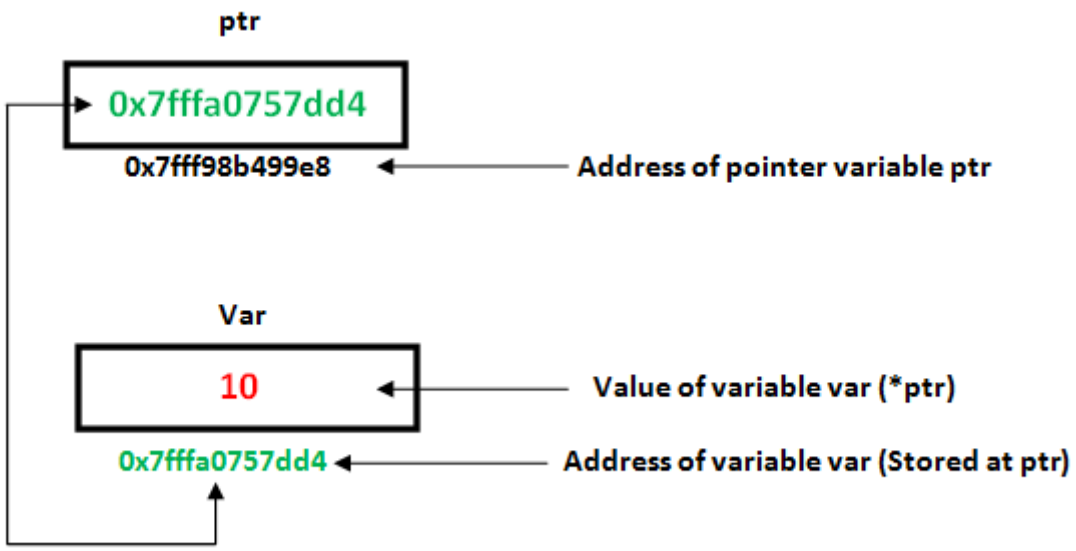
```
int arr[] = {1, 2, 3};
int* ptr = arr;    // points to arr[0]
ptr++;    // now points to arr[1]
```

6. **Dynamic Memory Allocation**
   Pointers are commonly used in dynamic memory allocation, where memory is allocated or deallocated during program execution. Functions like `new` and `delete` are used for this purpose.

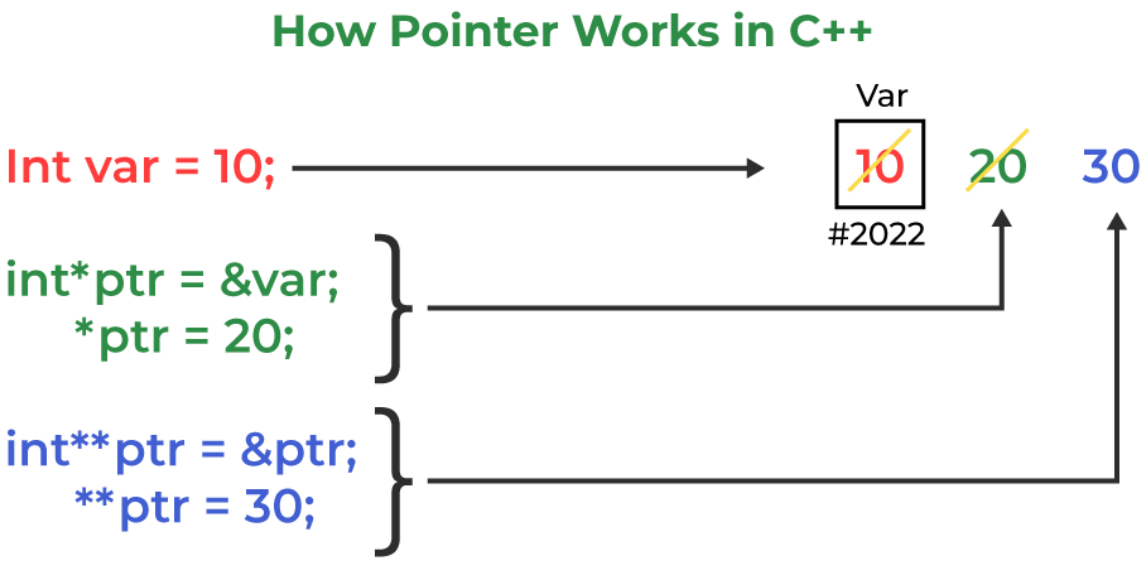
```
int *ptr = new int;    // Dynamically allocating memory for an integer
*ptr = 10;              // Assigning a value to the dynamically allocated memory
delete ptr;             // Deallocating the memory
```

   In this example, dynamic memory allocation is used to allocate memory for an integer and then deallocate it using the `delete` operator.

Visual Representation of Pointer



Working of Pointers



Here, `**ptr` is a pointer to `int *` & it's known as **pointer to pointer**.
So, `**ptr` stores address of another pointer variable of type `int *`.

Array Pointers

Array Pointers

Array pointers, also known as **pointers to arrays**, are pointers that point to the memory location of an array. Understanding how arrays and pointers interact in C++ is essential for handling arrays efficiently, especially when dealing with large amounts of data.

Key Concepts of Array Pointers

1. **Array to Pointer Relationship**
   In C++, the name of an array acts as a pointer to the first element of the array.
   This means that if you have an array say `arr`, then the name `arr` is essentially a constant pointer to the 1st element (`&arr[0]`).

```
int arr[5] = {10, 20, 30, 40, 50};
int* ptr = arr;    // ptr points to arr[0]
```

2. **Accessing Array Elements with Pointers**
   You can access elements of array using Pointer Arithmetic. Since the array name is a pointer to the 1st element, you can increment or decrement the pointer to move through the array.

```
int arr[3] = {1, 2, 3};
int* ptr = arr;

cout << *ptr << endl;    // Output: 1 (arr[0])
cout << *(ptr + 1) << endl;    // Output: 2 (arr[1])
cout << *(ptr + 2) << endl;    // Output: 3 (arr[2])
```

3. **Pointer to an Array**
   A pointer could also point to the entire array rather than just a single element. To declare a

pointer to an array, you need to specify the size of the array.

```
int arr[5] = {10, 20, 30, 40, 50};
int (*ptr)[5] = &arr;    // ptr is a pointer to the entire array
```

Here, `ptr` is a pointer to an array of 5 integers. To access elements of the array using this pointer, you can dereference it.

```
cout << (*ptr)[0] << endl;    // Output: 10
cout << (*ptr)[1] << endl;    // Output: 20
```

Difference between Array Pointer & Pointer to Array

- **Array Pointer (Pointer to the 1st element of an array)** :
  Declaring `int* ptr = arr;` means `ptr` points to the 1st element of the array.
  Pointer Arithmetic (`ptr + 1` , `ptr + 2` , etc.) will allow you to access subsequent elements.
- **Pointer to the entire array** :
  Declaring `int (*ptr)[5] = &arr;` means `ptr` points to the whole array, and dereferencing `(*ptr)` gives you access to the array. Here, Pointer Arithmetic will move the pointer across entire arrays (if there were more arrays).

Mechanics of Array Pointers in C++

```
int arr[] = {2, 3, 5, 7, 11};
cout << *arr << '\n';    // Prints 2
cout << *(arr + 1) << '\n';    // Prints 3
cout << 2[arr] << '\n';    // Prints 5
```

Behind the scenes, C++ also decodes `arr[i]` as `*(arr + i)`

Note : C++ also decodes `i[arr]` as `*(i + arr)`
Hence, `arr[i]` $\equiv$ `i[arr]` $\equiv$ `*(arr + i)` which is fairly weird

References

References

In C++, **References** provide an alternative name or alias for existing variable.
A reference is not a separate object, but another name for the same memory location.

References are often used in function parameters to allow functions to modify the arguments passed to them.

Also known as **Aliases**.

Key Points about References

1. **Declaration** : A reference is declared using the `&` symbol.

```
int a = 10;
int &ref = a;   // 'ref' is a reference to 'a'
```

2. **No Re-Assignment** : Once a reference is assigned to a variable, it cannot be changed to refer to another variable.

```
int b = 20;
ref = b;    // This does'nt change reference; instead, it assigns value of 'b' to 'a'
```

3. **Must be Initialised** : References must be initialised at the time of declaration. You can't create an uninitialised reference.

```
(int &) ref;    // Invalid, must initialize a reference
```

4. **Usage in Functions** : One of the main uses of references is in function parameters, allowing functions to modify the arguments passed to them without copying them.

```
void modify(int &x) {
   x = 100;
}

int num = 10;
modify(num);    // 'num' becomes 100 because 'x' is a reference to 'num'
```

5. **Const References** : A reference can be declared as `const`, meaning that it cannot be used to modify the referenced variable.

```
void print(const int &x) {
   cout << x;
}
```

NOTE :
`&` on LHS of `=` is actually for **References**
`&` on RHS of `=` is actually Address-of operator in **Pointers**

Difference from Pointers

- **References** must always be initialised, while **pointers** can be `nullptr`.
- **References** cannot be reassigned to refer to another object, while **pointers** can be changed to point to different objects.
- Accessing a **reference** is syntactically similar to accessing the variable itself, while **pointers** require dereferencing (`*`).

Templates

Templates

- A template in C++ is a feature that allows for the creation of generic functions or classes.
- It enables the writing of code in a way that it can handle any data type without the need to rewrite the same code for different data types.
- Templates allow for the creation of reusable code that can operate on different data types.

Why Templates are Used

1. **Code Reusability** : Templates enable the creation of functions and classes that work with any data type. This promotes code reuse and eliminates the need to duplicate code for different data types.
2. **Flexibility** : Templates provide flexibility by allowing the same code to be used with different data types, enhancing the adaptability and versatility of C++ programs.
3. **Type Safety** : Despite being generic, templates in C++ maintain type safety. The compiler performs type checking during instantiation to ensure type correctness.
4. **Performance** : Templates offer performance benefits by allowing the compiler to generate specialized code for each data type used, optimizing execution speed.

Code Example - 1

```
template <typename T>
template <class T>    // Alternate to above line of code

class Box {
public:
    T content;

    void setContent(T newContent) {
        content = newContent;
    }

    T getContent() {
        return content;
    }
};

int main() {
    Box<int> intBox;
    intBox.setContent(123);
    cout << intBox.getContent() << '\n';

    Box<string> stringBox;
    stringBox.setContent("Hello Templates");
    cout << stringBox.getContent() << '\n';

    return 0;
}
```

Explanation

- In this example, a class `Box` is defined using a template.
- The class `Box` has a member variable `content` of type `T`, and two member functions `setContent` and `getContent`.

- The `setContent` function sets the value of `content`, and the `getContent` function retrieves the value of `content`.
- The `main` function demonstrates the usage of the `Box` class with different data types (`int` and `std::string`).
- It creates instances of `Box` for `int` and `std::string`, sets their content, and then retrieves and prints the content.

Code Example - 2

```
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << max(10, 5) << '\n';      // The compiler generates int max(int, int)
    cout << max(10.5, 20.5) << '\n'; // The compiler generates double max(double,
double)
    return 0;
}
```

Explanation

- In this example, a template function `max` is defined, which takes two parameters of type `T` and returns the maximum of the two values.
- The `main` function demonstrates the usage of the `max` function with different data types (`int` and `double`).
- When `max` is called with integers (`max(10, 5)`), the compiler generates an instantiation of `max` for `int`.
- Similarly, when `max` is called with doubles (`max(10.5, 20.5)`), the compiler generates an instantiation of `max` for `double`.
- This shows how templates allow for the creation of generic functions that can work with different data types.

What is STL ?

The Standard Template Library (STL) is a powerful library that provides a set of common classes and functions, including algorithms and data structures like vectors, lists, stacks, queues and more. It is designed to handle generic programming, allowing developers to write code that works with any data type.

Key components of STL

1. **Algorithms**
   STL provides wide range of algorithms to perform common operations on data, such as searching, sorting, and manipulating data. These algorithms are implemented as template functions, meaning they can work with any container that supports iterators.

- rand()

rand()

In C++, `rand()` function is a part of the C standard library (`cstdlib`) and is used to generate **pseudo-random numbers**. While not technically part of the STL, it's commonly used in conjunction with STL containers & algorithms.

Key Points about `rand()`

1. **Function Definition** : The `rand()` function returns a pseudo-random integers in the range `[0, RAND_MAX]` , where value of `RAND_MAX` is library-dependent but is guaranteed to be at least 32767 on any standard library implementation.

   ```
   int rand();
   ```

2. **Seeding with `srand()`** : By default, the random number generator is seeded with the same value every time you run the program, which leads to the same sequence of random numbers.

   To generate different sequence of numbers, you can seed the generator using the `srand()` function.

   ```
   void srand(unsigned int seed);
   ```

   The common way to seed the random number generator is by using the current time, which ensures a different sequence of random numbers each time the program is run.

   ```
   #include <ctime>

   srand(time(0));    // Seed with current time
   rand();    // now everytime your code runs, 'rand()' gives you a new value
   ```

   `srand()` function is used to change the default seed in the `random` library.

NOTE :
In Python, `rand()` generates floating point numbers
In C++, `rand()` generates integers

Generating random numbers in a Range

Write a function that generates random number in range `[a, b]`

```
int random_ab(int &a, int &b) {
        return a + (ran() % (b - a + 1));
}
```

Limitations of `rand()`

- **Quality of Randomness** : `rand()` is a pseudo-random number generator (PRNG) and not suitable for cryptographic purposes or high quality randomness. It generates

numbers based on an algorithm that produces a deterministic sequence, which can be repeated if seeded with the same value.

- **Limited Range** : The maximum value that `rand()` can generate is limited by `RAND_MAX`. For generating random numbers in a much larger range, `rand()` might not be sufficient.

Modern Alternative `std::random` in C++11 and Later

C++11 Introduced a new random number generator framework in the `<random>` header, which provides better control and more advance random number generation mechanisms (e.g., different distributions and engines). This approach is more flexible and offers better randomness quality.

```
#include <random>
using namespace std;

int main() {
        // Create a random number generator
        random_device rd;    // Seed
        mt19937 gen(rd());    // Mersenne Twister engine
        uniform_int_distribution<> distr(1, 10);    // Range [1, 10]

        // Generate a random number in the range [1, 10]
        cout << "Random number between 1 and 10: " << distr(gen) <<
endl;

        return 0;
}
```

- random_shuffle() and shuffle()

random_shuffle()

The `random_shuffle()` function in C++ was used to randomly rearrange the elements in a given range,, producing a random permutation of the elements. It was part of the `<algorithm>` header in the C++ Standard Library.

Function Prototype

```
void random_shuffle(RandomIt first, RandomIt last);
void random_shuffle(RandomIt first, RandomIt last, RandomFunc rand);
```

- *first* : Iterator pointing to the beginning of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)
- *rand* (optional) : A function or callable object that provides random values

`random_shuffle` rearranged the elements randomly using an internal random number generator unless a custom generator was provided.

Example

```
#include <algorithm>
```

```
int main() {
        vector<int> v = {1, 2, 3, 4, 5};

        // Shuffle the vector using random_shuffle (deprecated in C++17)
        random_shuffle(v.begin(), v.end());

        return 0;
}
```

NOTE
However, it's important to note that `random_shuffle` was deprecated in C++14 and removed in C++17. In modern C++ (C++11 and later), the preferred function is `shuffle`, which is more flexible and requires an explicit random number generator.

Why `random_shuffle` was Deprecated and Removed ?

- `random_shuffle` used the `rand()` function internally, which was not a high-quality random number generator
- `rand()` has poor randomness and was implementation-dependent, which made it unsuitable for many modern applications that require good randomisation

Modern Replacement : `shuffle()`

Instead of `random_shuffle`, you should use `shuffle`, which allows for better control over the randomness by requiring an explicit random number generator (like one from the `<random>` library).

Function Prototype

```
template<class RandomIt, class URBG>
void shuffle(RandomIt first, RandomIt last, URBG&& g);
```

- *URBG* : Uniform Random Bit Generator (e.g., `default_random_engine`, `mt19937`)

Example

```
#include <random>
#include <algorithm>

int main() {
        vector<int> v = {1, 2, 3, 4, 5};

        // Initialise random number generator
        random_device rd;
        mt19937 g(rd());

        // Shuffle the vector using shuffle (modern C++)
        shuffle(v.begin(), v.end(), g)

        return 0;
}
```

Key Differences between `random_shuffle` and `shuffle`

- `shuffle` requires an explicit random number generator, making it more flexible and reliable for good randomness
- `random_shuffle` was tied to the poor-quality `rand()` function, which led to it's deprecation and removal

- next_permutation()

next_permutation()

In C++ STL, `next_permutation()` is a function that rearranges the elements in a range to the **next lexicographical permutation_** (or order) of those elements. If the current arrangement is the largest possible permutation, it rearranges the elements to the **smallest permutation** (sorted in ascending order).

Function Prototype

```
bool next_permutation(BidirectionalIt first, BidirectionalIt last);
```

- *first* : Iterator pointing to the first element of the range
- *last* : Iterator pointing to one past the last element of the range

Return value

- Returns `true` if the function was able to rearrange the elements into the next permutation
- Returns `false` if the function could not find a next permutation (i.e., if the current arrangement is the largest permutation possible)

NOTE : There exists 1 more similar STL algorithm, `prev_permutation()`

Key Points about `next_permutation()`

1. **Lexicographical Order** : The function generates permutations in lexicographical (dictionary) order
2. **Efficient Generation** : It efficiently generates the next permutation without generating all permutations at once, making it useful for algorithms requiring incremental permutations
3. **Sorting** : If the range is sorted in descending order (the largest lexicographical permutation), `next_permutation()` will sort the range in ascending order, returning `false`

Examples

`next_permutation()` with Strings

```
#include <algorithm>
#include <string>
using namespace std;

int main() {
    string s = "abc";
```

```
    do {
        cout << s << endl;
    } while (next_permutation(s.begin(), s.end()));

    return 0;
}
```

Output

```
abc
acb
bac
bca
cab
cba
```

After the last permutation (`cba`), the function returns `false` because there are no more possible permutations.

Sorting (Largest Permutation)

```
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3}; // Starting with the smallest permutation

    // Print all permutations until reaching the largest
    do {
        for (int x : v) {
            cout << x << " ";
        }
        cout << endl;
    } while (next_permutation(v.begin(), v.end()));

    // After the loop, v is now {3, 2, 1}, which is the largest
permutation
    cout << "After reaching largest permutation: ";
    for (int x : v) {
        cout << x << " ";
    }
    cout << endl;

    // Call next_permutation again
    next_permutation(v.begin(), v.end());

    // Print the rearranged vector
    cout << "Rearranged to smallest permutation: ";
    for (int x : v) {
        cout << x << " ";
```

```
    }
    cout << endl;

    return 0;
}
```

Output

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
After reaching largest permutation: 1 2 3
Rearranged to smallest permutation: 1 3 2
```

The `do-while` loop executes once before checking the condition. This means that if the last permutation is printed, the vector will be rearranged to the smallest permutation on the next call of `next_permutation()` inside the loop, leading to `123`, and then `132`.

Practical Applications

1. **Generating Permutations** : `next_permutation()` is widely used when you need to generate permutations in lexicographical order
2. **Solving Puzzles/Problems** : It can be used to solve combinatorial problems that require exploring all possible orders of elements
3. **Brute Force Solution** : Useful in brute force algorithms that need to try all possible arrangements of elements, such as solving permutation-based puzzles or optimisation problems

Self Implementation of `next_permutation()`

Problem Link : 31. Next Permutation

Platform : #Platform-LeetCode

Difficulty : #Difficulty-Medium

Approach-1
Generate all possible permutations using Recursion, then sort them in increasing order & return the next index

```
class Solution {
public:
    void generatePermutations(vector<int> &nums, vector<vector<int>>
&permutations, int ind) {
        if(ind == nums.size()) {
            permutations.push_back(nums);
            return;
        }
```

```
        for(int i=ind; i<nums.size(); i++) {

            // Avoid duplicate permutations when elements repeats
            // For better understanding, dry run {1, 1, 2, 3}
            if(i == ind || (i != ind && nums[ind] != nums[i])) {
                swap(nums[ind], nums[i]);
                generatePermutations(nums, permutations, ind+1);
                swap(nums[ind], nums[i]);
            }
        }
    }

    void nextPermutation(vector<int>& nums) {
        vector<vector<int>> permutations;

        // Generate all permutations in sorted order
        generatePermutations(nums, permutations, 0);
        sort(permutations.begin(), permutations.end());

        for(auto it:permutations) {
            for(auto p:it) cout << p << " ";
            cout << endl;
        }

        // Linear Search to find where does 'nums' lie
        int n = permutations.size(), ind;
        for(int i=0; i<n; i++) {
            if(permutations[i] == nums) {
                ind = i;
                break;
            }
        }

        // Next index permutation will be answer
        // If next index doesn't exist, then 1st will be answer
        nums = permutations[(ind+1) % permutations.size()];
    }
};
```

TC : $O(n!) + O(n! * \log(n!)) + O(n!)$
SC : $O(n! * n)$

Approach-2
Observations,

1. Next elements of a dictionary tend to have longer prefix match

Algorithm

1. Try getting longest prefix match possible, by finding the break point (increasing 0 to i+1 & decreasing from i+1 to n-1)

2. Once you figure out break point, after the longest prefix try putting someone at i who is slightly greater than current guy
3. Fill the remaining portion i+1 to n-1 as small as possible, by filling in sorted order
   Eg : {2, 1, 5, 4, 3, 0, 0}
   `i` will be pointing to value 5
   {2, 3, , , , , _}
   {2, 3, 0, 0, 1, 4, 5}
4. {1, 2, 3, 4, 5} dip right before the last element, so we know dip highest could be found at (n-2)th index
5. {5, 4, 3, 2, 1} whole decreasing, so no dip (as it's the last permutation)

```
class Solution {
public:
    void nextPermutation(vector<int>& nums) {
        int n = nums.size();

        // Find the dipIndex
        int dipIndex = -1;
        for(int i=n-2; i>=0; i--) {
            if(nums[i] < nums[i+1]) {
                dipIndex = i;
                break;
            }
        }

        // If there's no dip, then it would be lexicographically the
biggest permutation, so return smallest one
        if(dipIndex == -1) {
            reverse(nums.begin(), nums.end());
            return;
        }

        // Find someone ahead which is slightly greater than element at
dipIndex
        // Since right portion is decreasing & we need smallest, so
start traversing from back
        for(int i=n-1; i>=0; i--) {
            if(nums[i] > nums[dipIndex]) {
                swap(nums[i], nums[dipIndex]);    // Swapping let's [i+1
... n-1] stay in decreasing order
                break;
            }
        }

        // Fill the remaining portion as small as possible, by filling
it in sorted order
        // Since that part is decreasing, so we just reverse it
        reverse(nums.begin() + dipIndex+1, nums.end());
    }
};
```

TC : $O(N)$
SC : $O(1)$ (Modifying the given array)

- lower_bound() and upper_bound()

lower_bound() and upper_bound()

In C++ STL, `lower_bound` and `upper_bound` are 2 functions used to find elements or positions in sorted sequences (like `arrays`, `vectors`, or other sorted containers). Both functions are part of the `<algorithm>` header and are commonly used with containers that support **random access iterators**.

lower_bound()

The `lower_bound()` function returns an iterator pointing to the **first element** in the range that is **not less than** the specified value (i.e., it is greater than or equal to the given value).

Syntax :

```
RandomIt lower_bound(RandomIt first, RandomIt last, const T& value);
```

- *first* : Iterator pointing to the start of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)
- *value* : The value to search for

Example :

```
int main() {
        vector<int> v = {1, 3, ,3, 7, 9};
        auto it = lower_bound(v.begin(), v.end(), 3);

        // Print index and value of the lower bound
        cout << "LB of 3 is at index: " << it - v.begin() << ", value: "
<< *it << endl;
        // Output: LB of 3 is at index: 1, value: 3

        return 0;
}
```

In this example, `lower_bound` returns the iterator to the 1st occurrence of 3, or the position where 3 would be inserted it it's not present.

Characteristics of `lower_bound` :

- *Time Complexity* : $O(logN)$, where N is number of elements between `first` and `last` (binary search)
- If the value is not present, it returns an iterator to the position where the value would be inserted to maintain the sorted order
- It's index gives you the number of elements less than the specified value
- If lower bound doesn't exist, then it returns `v.end()` or `arr + n`, which gives us index as n (i.e. size)

upper_bound()

The `lower_bound()` function returns an iterator pointing to the **first element** in the range that is **greater than** the specified value.

Syntax :

```
RandomIt upper_bound(RandomIt first, RandomIt last, const T& value);
```

- *first* : Iterator pointing to the start of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)
- *value* : The value to search for

Example :

```
int main() {
        vector<int> v = {1, 3, ,3, 7, 9};
        auto it = upper_bound(v.begin(), v.end(), 3);

        // Print index and value of the lower bound
        cout << "UB of 3 is at index: " << it - v.begin() << ", value: "
<< *it << endl;
        // Output: UB of 3 is at index: 1, value: 3

        return 0;
}
```

In this example, `upper_bound` returns the iterator to the 1st element greater than 3, which is 7.

Characteristics of `upper_bound` :

- *Time Complexity* : $O(logN)$, where N is number of elements between `first` and `last` (binary search)
- If the value is not present, it returns an iterator to the position where a larger value would occur in the sorted order
- It's index gives you the number of elements less than or equal to the specified value
- If upper bound doesn't exist, then it returns `v.end()` or `arr + n`, which gives us index as n (i.e. size)

Key Differences between `lower_bound()` and `upper_bound()`

- `lower_bound` returns the iterator to the 1st element that is **greater than or equal** to the value
- `upper_bound` returns the iterator to the 1st element that is **strictly greater** than the value

NOTE : Both functions assume that the range `[first, last)` is sorted

- reverse()

reverse()

In C++ STL, the `reverse()` function is used to reverse the order of elements in a given range. This function is part of the `<algorithm>` header and works with any container that supports bidirectional or random access iterators (such as `vector`, `deque`, `array`, etc.)

Function Prototype

```
void reverse(BidirectionalIt first, BiDirectionalIt last);
```

- *first* : Iterator pointing to the beginning of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)

Time Complexity : $O(N)$, where N is the number of elements in range `[first, last)`.

Example

```
#include <algorithm>

int main() {
        vector<int> v = {1, 2, 3, 4, 5}

        // Reversing the entire vector
        reverse(v.begin(), v.end());

        // Reversing a part of the vector (from index 1 to 4)
        reverse(v.begin() + 1, v.begin() + 5);

        return 0;
}
```

How `reverse()` works ?

- The function works by swapping elements starting from both ends of the range and moving towards the center.
- It requires the container to have bidirectional iterators, which allow moving forward and backward in the range (containers like `list`, `vector`, `deque` support this).

- sort() and Lambda Comparators

sort()

The `sort()` function in C++ STL is a powerful algorithm used to sort elements in a range (such as arrays or containers like `vector`, `deque`, etc.). It is a part of the `<algorithm>` header, and it's highly efficient, typically implemented using the **Introsort** algorithm, which is a hybrid sorting algorithm combining Quick Sort, Heap Sort, and Insertion Sort (because these 3 don't require extra spaces).

Function Prototype

```
void sort(RandomIt first, RandomIt last);
```

- *first* : Iterator pointing to the beginning of the range
- *last* : Iterator pointing to the end of the range (the position one past the last element)

Key Points about `sort()`

1. **Random Access Iterators** : `sort()` requires the iterators to be random access (e.g., for `vector`, `array`, `deque`). Containers like `list` and `forward_list` that do not provide random access iterators cannot use `sort()`.
2. **Default Sorting (Ascending order)** : By default, `sort()` sorts elements in ascending order based on the `<` operator.

> - For integers, strings, floating-point numbers, etc., it uses the built-in comparison operators.
> - For custom data types, you need to overload the `<` operator or provide a custom comparator.

3. **Time Complexity** : Average-case and worst-case time complexity is **O(NlogN)**, where **N** is the number of elements to be sorted.
4. **Custom Comparators** : You can provide a custom comparator (a function or a function object) to define the sorting criterion. This is useful when you want to sort in descending order or use custom rules for sorting.

Examples of using `sort()`

Sorting in Ascending Order

```
sort(v.begin(), v.end());    // Sorting an vector
sort(v, v + n);    // Sorting an array
```

Partial Sorting (using `sort()` on a Range)

```
// Sort elements from idnex 1 to 4 (excluding index 4)
sort(v.begin() + 1, v.begin() + 4);
```

Sorting with Custom Comparator (User-defined Objects)

```
struct Person {
        string name;
        int age;
}

// Custom comparator to sort by age
bool compareByAge(const Person &a, const Person &b) {
        return a.age < b.age;    // Sort in ascending order of age
}

int main() {
        vector<Person> people = {{"Kartik", 21}, {"Cat", 19}, {"Mayank",
31}, {"Mohit", 18}};

        // Sort people by age
        sort(people.begin(), people.end(), compareByAge);

        return 0;
}
```

Lambda Expressions

In C++11 and Later, you can create a comparator **inline** using **lambda expressions**. This makes it easier to write short, simple comparators without needing to define a separate function. Lambda expressions provide a concise way to create anonymous functions directly at the place where you need them.

Syntax of Lambda Expressions

A Lambda expression has the following general form:

```
[ capture ] ( parameters ) -> return_type {body}
```

- **Capture** : Specifies which variables from thee surrounding scope should be accessible in the lambda.
  Common captures include:
  - `[ ]` (no capture)
  - `[=]` (capture all variables by value)
  - `[&]` (capture all variables by reference)
  - `[x]` (capture only `x` by value), `[&x]` (capture only `x` by reference)
- **Parameters** : The function arguments (optional)
- **Return Type** : Can be inferred (optional)
- **Body** : The function body (code to execute)

Examples of using Lambda Expressions

Sorting Custom Objects with a Lambda Comparator

```
struct Person {
        string name;
        int age;
}

int main() {
        vector<Person> people = {{"Kartik", 21}, {"Cat", 19}, {"Mayank",
31}, {"Mohit", 18}};

        // Sort people by age using a lambda expression
        sort(people.begin(), people.end(), [](const Person &a, const
Person &b){
                return a.age < b.age;    // Sort in ascending order of
age
        });

        return 0;
}
```

Sorting by Multiple Criteria using a Lambda

```
struct Person {
        string name;
        int age;
}

int main() {
        vector<Person> people = {{"Kartik", 21}, {"Cat", 19}, {"Mayank",
31}, {"Mohit", 18}};

        // Sort people by age using a lambda expression
```

```
        sort(people.begin(), people.end(), [](const Person &a, const
Person &b){
                if(a.age == b.age) {
                        return a.name < b.name;    // If ages are equal,
sort by name
                }
                return a.age < b.age;    // Sort in ascending order of
age
        });

        return 0;
}
```

Advantages of using Lambda Functions for Comparators

1. **Conciseness** : Inline lambda functions eliminate the need to write separate comparator functions, making your code more compact and readable.
2. **Ease of Use** : For simple comparators, lambdas are quick to write and don't require additional function definitions.
3. **Clarity** : The sorting logic is placed where it is used, improving the code's readability by keeping relevant parts close together.

- __gcd() and gcd()

__gcd()

In C++ STL, `__gcd()` is a function used to compute the **Greatest Common Divisor (GCD)** of 2 integers. It is a part of the `<algorithm>` header.

Function Prototype

```
int __gcd(int a, int b);
```

- *Parameters* : Two integers `a` and `b`
- *Returns* : The largest positive integer that divides both `a` and `b`

It uses **Euclidean algorithm** under the hood, which is very efficient for calculating GCD.

NOTE
However, from C++17 onwards, the function `gcd()` is officially introduced, which should be preferred over `__gcd()` for better standard compliance.

gcd()

`gcd()` is the official GCD function introduced in C++17, part of the `<numeric>` header.

Function Prototype

```
template <class T>
T gcd(T a, T b);
```

- *Template Type* : `T` can be any integral type (`int` , `long` , `long long` , etc.)
- *Parameters* : Two integers `a` and `b` of the same type

- *Returns* : The greatest common divisor of `a` and `b`, as a value of type `T`

We should prefer `gcd()` for modern C++ code because it's standardised and works for different integral types.

- min_element() and max_element()

min_element() and max_element()

In C++ STL, `min_element` and `max_element` are functions that are used to find the **minimum** and **maximum elements**, respectively, in a given range of elements (like arrays, vectors, or lists). Both functions are part of the `<algorithm>` header.

Key Points

- Both functions work with any container that provides forward iterators, such as `vector`, `list`, or `arrays`
- They both operate in $O(N)$ time complexity
- Custom comparators can be used to define how elements should be compared

min_element()

Function Prototype

```
template<class ForwardIt> ForwardIt min_element(ForwardIt first,
ForwardIt last);
```

- *first* : An iterator pointing to the first element of the range
- *last* : An iterator pointing to one past the last element of the range

Return Value

- Returns an iterator pointing to the smallest element in the specified range
- If the range is empty, it returns `last`

max_element()

Function Prototype

```
`template<class ForwardIt> ForwardIt max_element(ForwardIt first,
ForwardIt last);`
```

- *first* : An iterator pointing to the first element of the range
- *last* : An iterator pointing to one past the last element of the range

Return Value

- Returns an iterator pointing to the largest element in the specified range
- If the range is empty, it returns `last`

Example with Custom Comparators

Both functions can also accept a custom comparator to determine how to compare the elements

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
using namespace std;

int main() {
    vector<int> numbers = {-5, 3, -8, 1, 4};

    // Finding the minimum element based on absolute values
    auto min_it = min_element(numbers.begin(), numbers.end(), [](int a,
int b) {
            return abs(a) < abs(b);
        });

    if (min_it != numbers.end()) {
        cout << "The minimum element based on absolute value is: " <<
*min_it << endl;
    }

    // Finding the maximum element based on absolute values
    auto max_it = max_element(numbers.begin(), numbers.end(), [](int a,
int b) {
            return abs(a) < abs(b);
        });

    if (max_it != numbers.end()) {
        cout << "The maximum element based on absolute value is: " <<
*max_it << endl;
    }

    return 0;
}
```

Output

```
The minimum element based on absolute value is: 1
The maximum element based on absolute value is: -8
```

NOTE

1. For `min_element`,
   By using `abs(a) < abs(b)`, you are asking the function to return the element for which the absolute value is the smallest. Hence, it correctly identifies the minimum based on absolute values.
2. For `max_element`,
   Again, using `abs(a) < abs(b)` is logical in this context. It means that if the absolute value of `a` is less than that of `b`, then `a` should be considered "less" than `b` in terms of absolute value. Thus, the function will return the element that has the greatest absolute value as the maximum.

Why Not `abs(a) > abs(b)` for `max_element`?
If you were to use `abs(a) > abs(b)` for `max_element`, it would cause the function to

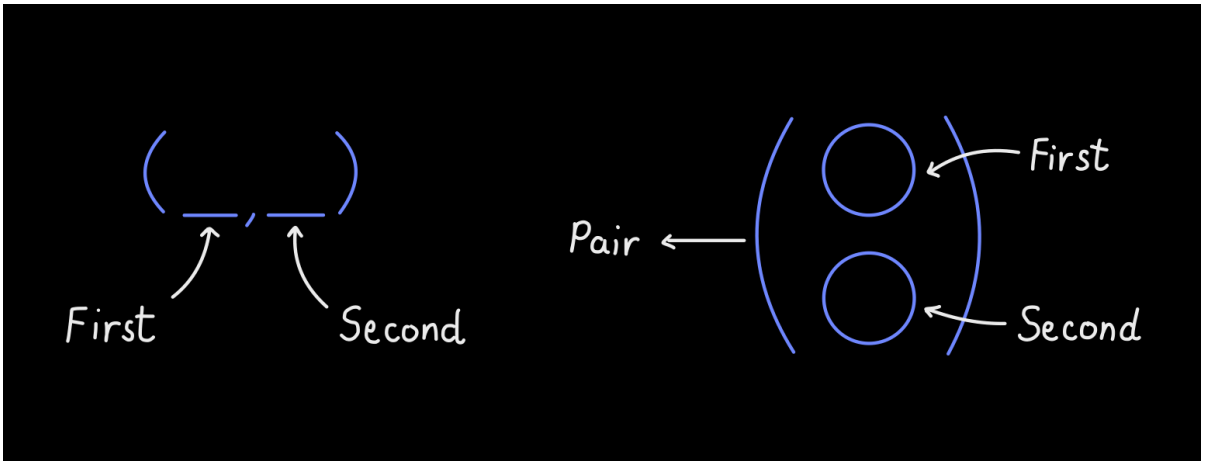
> behave incorrectly. Specifically, it would end up returning the **minimum** absolute value instead of the maximum because you would be saying that an element is "greater" when its absolute value is greater.

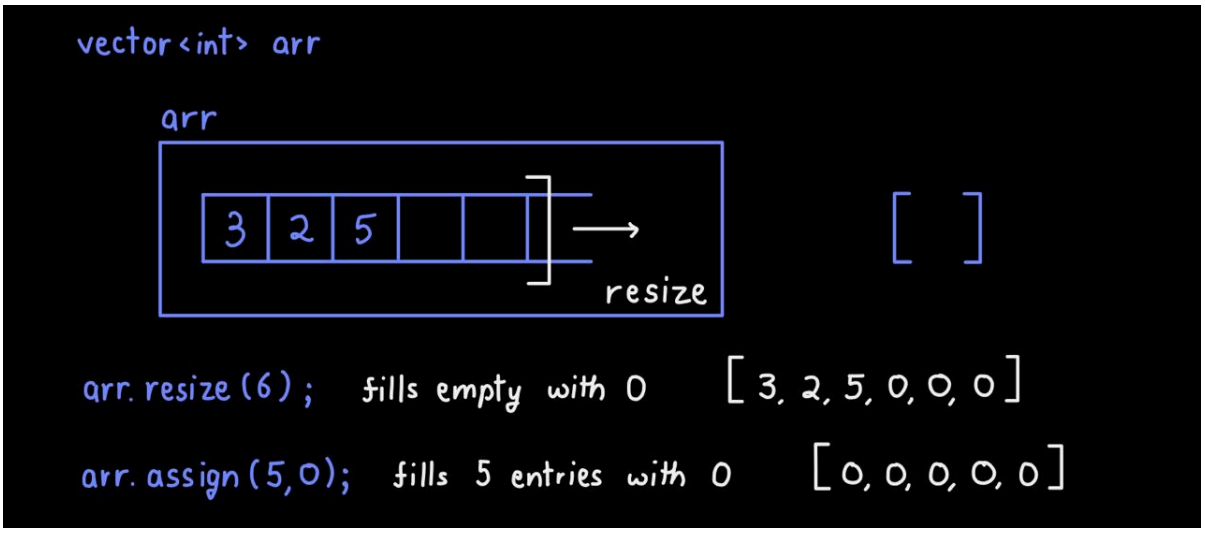
For reference, use website [cplusplus]

2. **Containers**
   These are data structures like arrays, vectors, linked lists etc. that store collections of objects.

   All the containers in the C++ Standard Template Library (STL) are defined as templates.
   This allows them to be **generic** and work with any data type.

   We aim to create mental models for how the STL containers should look in one's mind, making it easier to visualise problems and their solutions mentally.

> Mental Models of various Containers
>
> # Pair
>
> A `pair` in C++ STL is used to store two values, which can be of the same or different types.
>
> **Real-life Example**
> Storing coordinates (x, y): `pair<int, int>` can be used to store the coordinates of a point in a 2D plane.
>
> **Mental Model**
>
> 
>
>
> # Vector
>
> A `vector` is a dynamic array that can grow and shrink in size as needed. It allows random access to its elements.
>
> **Real-life Example**
> Storing a list of student scores: `vector<int>` can be used to store the test scores of a class of students.
>
> **Mental Model**



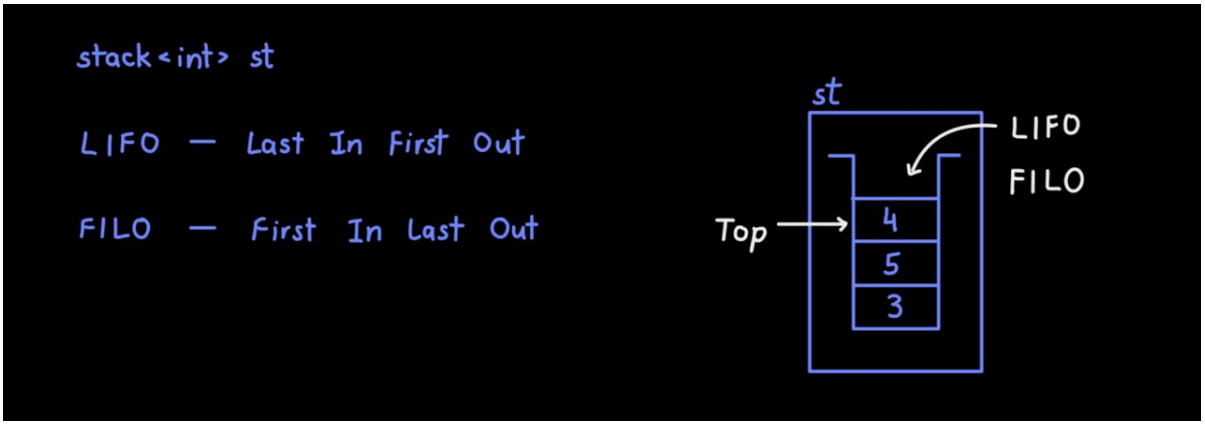
NOTE
For storing 2-3 values, use pair. Otherwise, use vector
No need of using tuple

Stack

A `stack` follows the **LIFO (Last In, First Out)** principle, where the last element added is the first one to be removed.

Real-life Example
Undo operation in text editors: Each action (such as typing or deleting) is pushed onto a stack, and undo operations pop the actions from the stack.

Mental Model

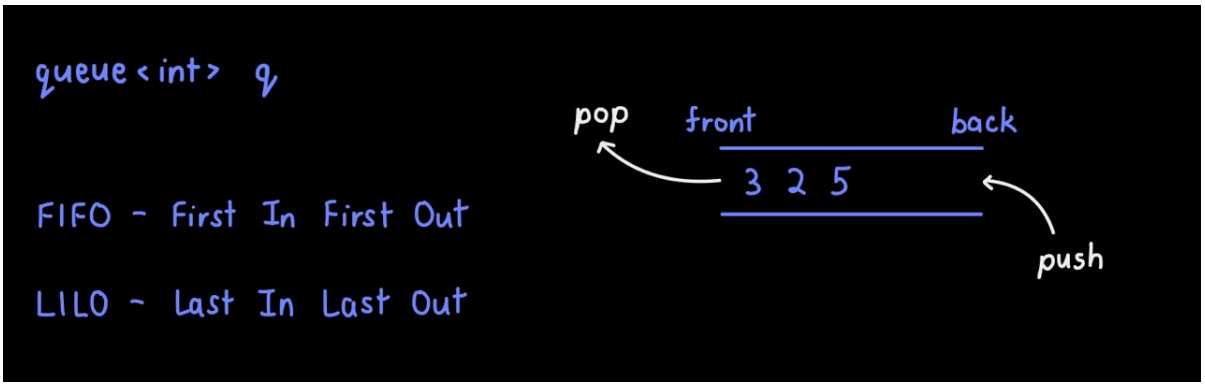


Queue

A `queue` follows the **FIFO (First In, First Out)** principle, where the first element added is the first one to be removed.

Real-life Example
Customer service line: Customers are served in the order they arrive, which can be modeled using a `queue<string>`.

Mental Model

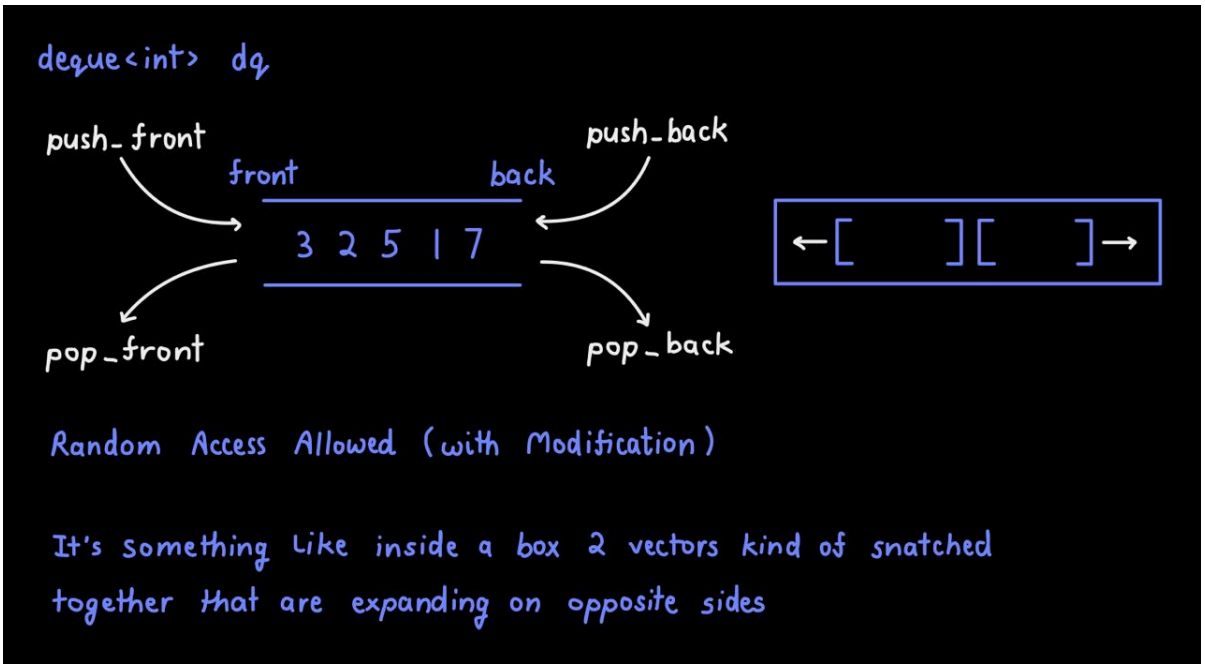


Deque

A `deque` is a container where you can add or remove elements from both ends (front or back). It supports both LIFO and FIFO operations.

Real-life Example
Browser history navigation: A `deque` can be used to store URLs, where you can navigate forward and backward in your browsing history.

Mental Model

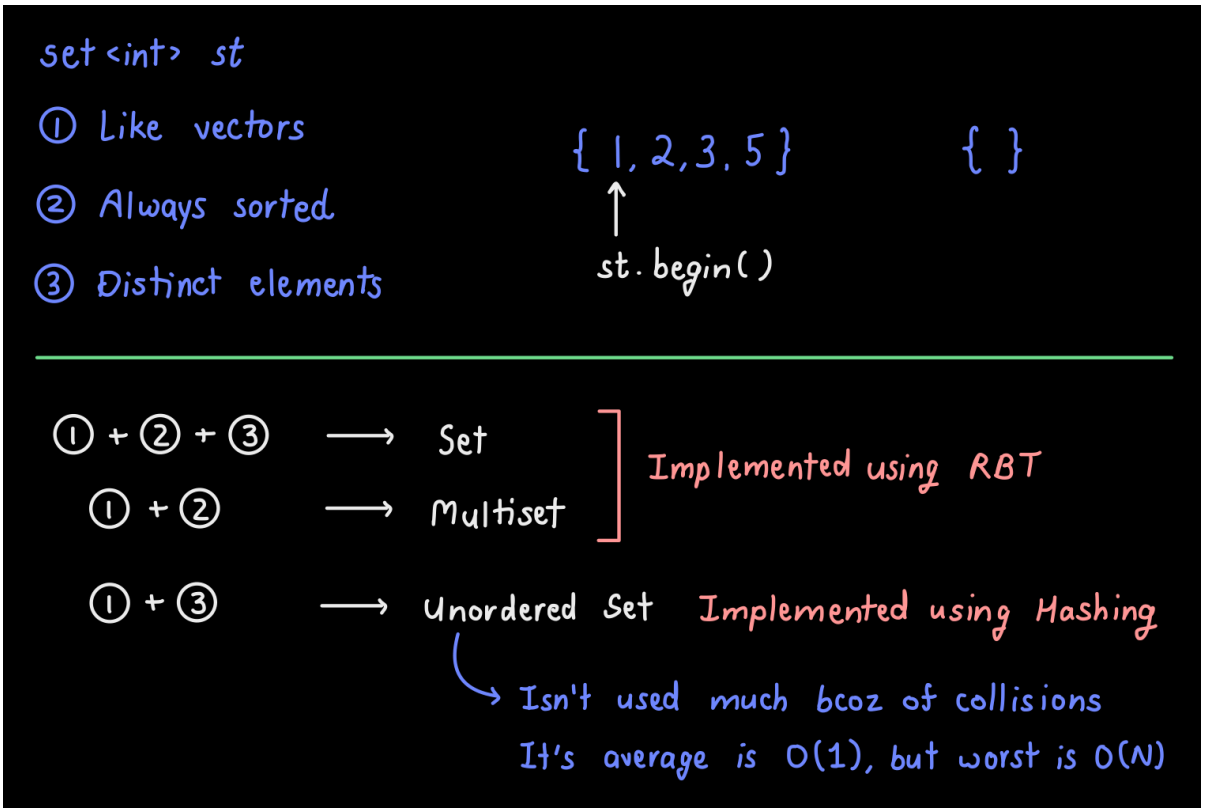


Set

A `set` is a container that stores unique elements in a sorted order. It does not allow duplicates.

Real-life Example
Storing a collection of unique email addresses: A `set<string>` can store email addresses to ensure no duplicates are added.

Mental Model



NOTE

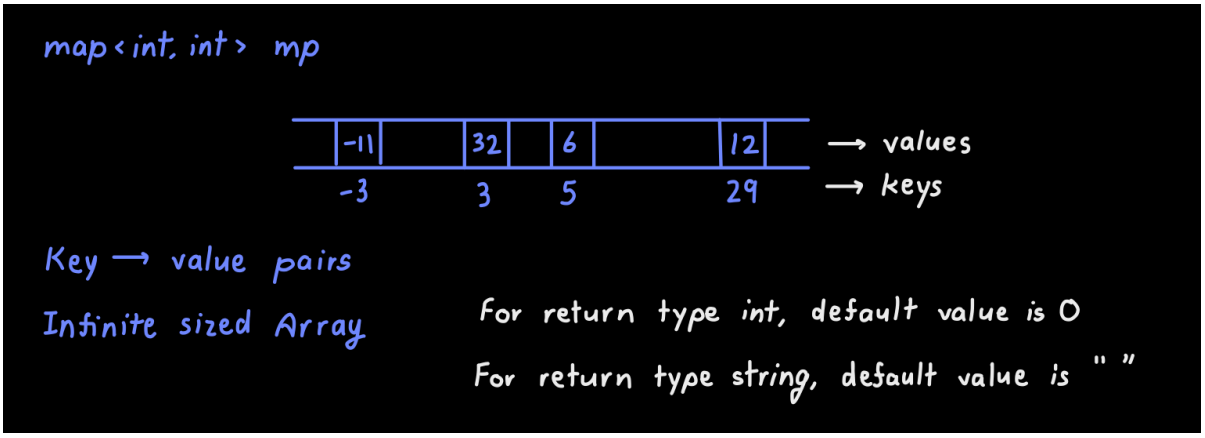
- In general, whenever we have curly braces, we loose the feature of indexing (random access)
- Although set is not subscriptable, but it's iterable

Map

A `map` stores key-value pairs, where each key is unique, and is used to look up values based on keys. It stores elements in sorted order of keys.

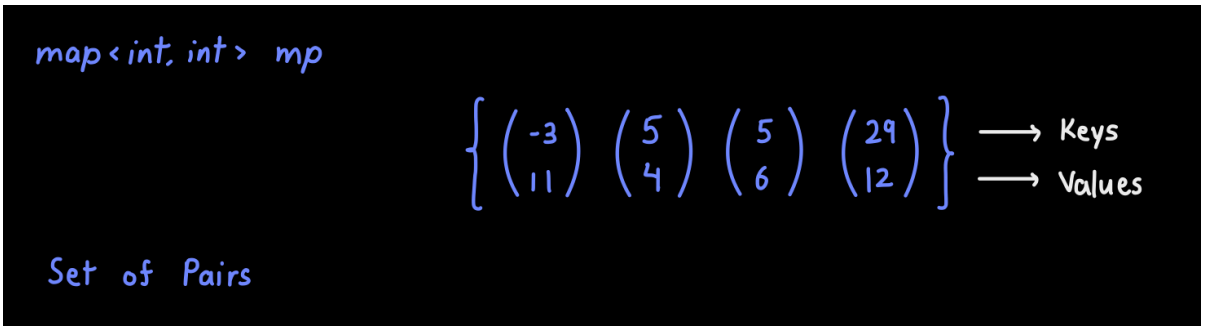
Real-life Example
Phone directory: A `map<string, string>` can be used to store names (keys) and their corresponding phone numbers (values).

Mental Model - 1 (Infinite Sized Array)



It's not like intermediate indexes are getting created in reality, it's just a mental model (how it's behaving and not how it's actually implemented)

Mental Model - 2 (Set of Pairs)



Since, it's a set, it'll always remain sorted, hence map are always sorted by the keys (unique)
All keys are unique because if you overwrite something, it'll get overwritten in value also

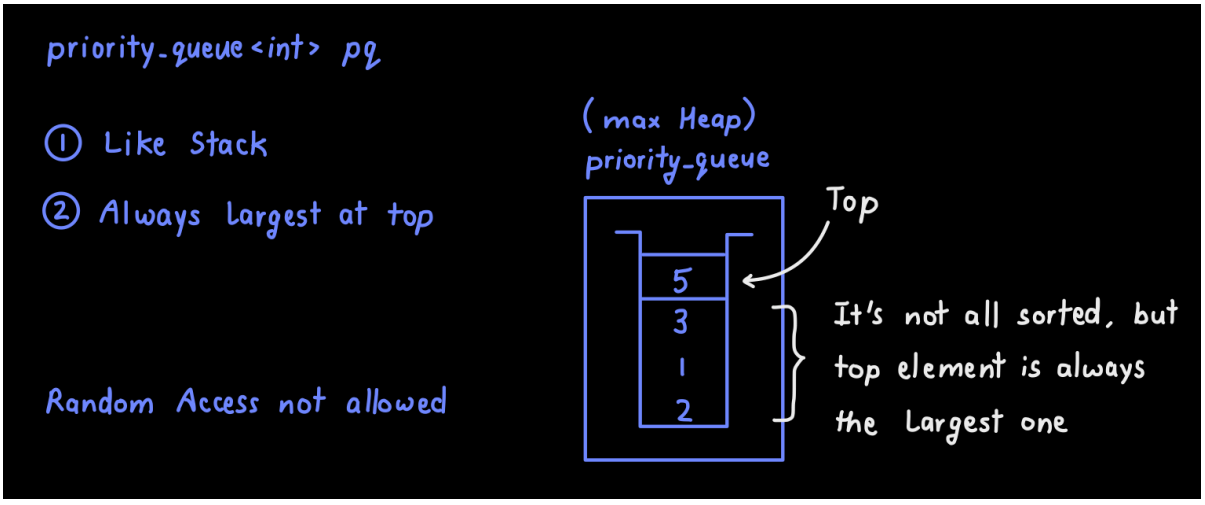
NOTE : Size is equal to number of keys (as they're unique)

Priority Queue

A `priority_queue` is a special type of queue where elements are ordered by their priority. By default, it's a max-heap, where the largest element is dequeued first.

Real-life Example
Task scheduling system: In a task scheduler, tasks with higher priority are processed before tasks with lower priority.

Mental Model



Pending

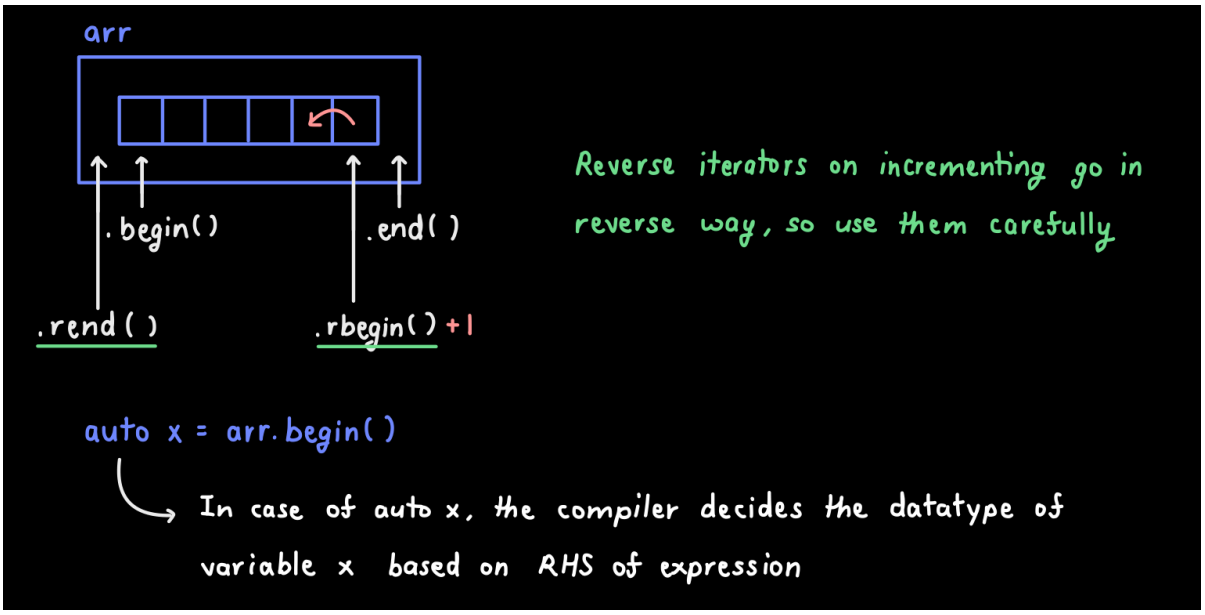
List : At time of Linked List
Bitset : At time of Bit Manipulation

For efficient traversal in various container types, we require
Iterators

Iterators

Iterators in C++ Standard Template Library (STL) are objects that act as pointers to elements within containers, allowing traversal and manipulation of the container's elements. They are a crucial part of the STL as they provide a uniform way to access elements of different container types (like `vector`, `set`, `map`, etc.).

Mental Model



Looping using Iterators

```
vector<int> arr = {3, 2, 5, 7};

for(auto x:arr) {
        cout << x << " ";
}
cout << '\n';

// Alternate way
for(int x:arr) {
        cout << x << " ";
}
```

Key Concepts of Iterators

1. **Pointer-like Behavior**: Iterators behave like pointers in that you can dereference them to access the elements they point to. They can be incremented (or decremented, depending on the iterator type) to move through the container.
2. **Container Agnostic**: Iterators provide a generic interface to traverse containers, meaning the same logic can be applied to different types of containers (e.g., `vector`, `set`, `map`) without needing container-specific traversal logic.
3. **Iterator Categories**: There are different types of iterators, each with different levels of capabilities:
   - *Input Iterators*: Can only read the data, and are used for single-pass algorithms.
   - *Output Iterators*: Can only write data.
   - *Forward Iterators*: Can read/write data and can move forward (used in `forward_list`).
   - *Bidirectional Iterators*: Can move forward and backward (used in `list`, `set`, `map`).

- *Random Access Iterators*: Can move directly to any element (used in `vector`, `deque`, `array`).

4. **Iterator Functions**: Iterators are used with common STL functions such as:
   - `begin()`: Returns an iterator pointing to the first element.
   - `end()`: Returns an iterator pointing one past the last element (not the last element itself).
   - `rbegin()`: Returns a reverse iterator starting from the last element.
   - `rend()`: Returns a reverse iterator pointing to one before the first element.

Types of Iterators

1. **Forward Iterator**:
   Can only move in one direction (forward).
   Can be used with containers like `forward_list`.

```
forward_list<int> fl = {1, 2, 3};
forward_list<int>::iterator it = fl.begin();
```

2. **Bidirectional Iterator**:
   Can move both forward and backward.
   Used with containers like `list`, `set`, `map`.

```
list<int> lst = {1, 2, 3};
list<int>::iterator it = lst.begin();
```

3. **Random Access Iterator**:
   Can directly access any element, like an array index (`it + 3` for the 4th element).
   Used with containers like `vector`, `deque`, and `array`.

```
vector<int> vec = {1, 2, 3, 4, 5};
vector<int>::iterator it = vec.begin();
cout << *(it + 2);  // Outputs 3
```

4. **Reverse Iterator**:
   Used to traverse containers from the end to the beginning.
   Common in containers like `vector`, `deque`, `list`.

```
vector<int> vec = {1, 2, 3, 4, 5};
for (vector<int>::reverse_iterator rit = vec.rbegin(); rit !=
vec.rend(); ++rit) {
        cout << *rit << " ";  // Output: 5 4 3 2 1
}
```

Advantages of Using Iterators

1. **Container Independence**: Iterators provide a common way to traverse different types of containers, allowing algorithms to be written in a container-agnostic manner.
2. **Efficiency**: Iterators are lightweight and efficient for accessing elements, especially in containers that don't support direct indexing (e.g., `list`, `set`).

3. **Flexibility**: They offer flexibility in traversing and modifying containers, including forward and backward traversal, and even random access in certain cases.
4. **STL Algorithm Compatibility**: Many STL algorithms (`std::sort`, `std::find`, etc.) rely on iterators, making them central to how STL operates.

Various Container Types in STL

- Vector

Vector

A **vector** is a dynamic array that can resize itself automatically when elements are added or removed. It is part of the sequence container category and is widely used due to its flexibility, efficiency, and simplicity, and it provides random access to its elements.

Real-Life Example

Let's consider a scenario where you want to store and manage a list of students' scores in a course. A vector would be an excellent choice for this task because:

- *Dynamic Sizing*: As students enroll or withdraw from the course, the vector can automatically adjust its size to accommodate the changing number of scores.
- *Efficient Access*: Vector provides constant time complexity for random access, making it efficient to retrieve a specific student's score.

Example Code

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> grades;

    // Adding grades to the vector
    grades.push_back(85);
    grades.push_back(90);
    grades.push_back(78);
    grades.push_back(92);

    // Accessing and printing grades
    cout << "Grades: ";
    for (int i = 0; i < grades.size(); i++) {
        cout << grades[i] << " ";
    }
    cout << endl;

    // Removing the last grade
    grades.pop_back();

    // Printing updated grades
    cout << "Updated Grades: ";
    for (int grade : grades) {
        cout << grade << " ";
    }
    cout << endl;
```

```
    return 0;
}
```

Key Characteristics of Vector

- **Dynamic Size**: Vectors can grow and shrink in size as elements are added or removed.
- **Random Access**: They allow constant-time access to elements using the subscript operator (`[]`).
- **Contiguous Memory**: Elements are stored in contiguous memory locations, which makes them cache-friendly and allows for efficient iteration.
- **Support for Iterators**: Vectors support iterators, enabling easy traversal and manipulation of elements.

Important Modifiers of Containers Vector

- `assign()` : Time Complexity - O(n), where n is the number of elements assigned.
- `push_back()` : Amortized Time Complexity - O(1) (constant time on average), though individual insertions might take O(n) in the worst case when resizing the vector.
- `pop_back()` : Time Complexity - O(1), constant time.
- `insert(position, value)` : Time Complexity - O(n), where n is the number of elements inserted. Inserting in the middle or at the beginning requires shifting elements.
- `erase(position)` : Time Complexity - O(n), where n is the number of elements erased. Erasing in the middle or at the beginning also requires shifting elements.
- `swap()` : Time Complexity - O(1), constant time. Swapping involves changing internal pointers, not actual element movement.
- `clear()` : Time Complexity - O(n), where n is the number of elements in the vector. This function removes all elements and may involve deallocating memory.
- `emplace()` : Time Complexity - O(n), where n is the number of elements after the insertion point. Elements after the insertion point may need to be shifted.
- `emplace_back()` : Amortized Time Complexity - O(1) (constant time on average), though individual insertions might take O(n) in the worst case when resizing the vector.
- `resize(new_size)` : Changes the size of the vector to `new_size` .
- `reserve(new_capacity)` : Reserves space for at least `new_capacity` elements, improving performance when you know in advance how many elements will be added.

Underlying Container of Vector

Vectors are implemented as dynamic arrays, which means they manage their own memory and can resize themselves as needed. When the size exceeds the current capacity, a new larger array is allocated, and the existing elements are copied to this new array.

Applications of Vector

1. **Dynamic Arrays**: Vectors are widely used to implement dynamic arrays where the number of elements is not known beforehand.
2. **Data Storage**: They are useful for storing collections of data, such as lists of records or a series of measurements.
3. **Matrix Representation**: Vectors can be used to represent matrices in multi-dimensional arrays by using vectors of vectors.

4. **Sorting and Searching**: Vectors provide an easy way to perform sorting and searching operations due to their random access capabilities.

- Stack

Stack

Stack is a data structure that follows the Last In, First Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed. Elements are added and removed from the same end, typically referred to as the "top" of the stack.

Real-life Example

Imagine a stack of plates in a cafeteria. Plates are stacked one on top of another. When a new plate is added, it goes on the top of the stack. When someone needs a plate, they take it from the top. This is a real-life example of a stack where the last plate added (last in) is the first one used (first out).

Example Code

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    // Declaration and Initialization
    // Time Complexity: O(1) for initialization
    stack<int> myStack;

    // Pushing elements onto the stack
    // Time Complexity: O(1) for push
    myStack.push(10);
    myStack.push(20);
    myStack.push(30);

    // Displaying the top element
    // Time Complexity: O(1) for top
    cout << "Top element: " << myStack.top() << endl;

    // Popping the top element
    // Time Complexity: O(1) for pop
    myStack.pop();

    // Checking if the stack is empty
    // Time Complexity: O(1) for empty
    if (!myStack.empty()) {
        // Time Complexity: O(1) for size
        cout << "Size of stack: " << myStack.size() << endl;
    }

    return 0;
}
```

Key Characteristics of Stack

1. **LIFO (Last In, First Out)**: The most recently added element is the first to be removed.

2. **Container Adapter**: The `stack` is a container adapter, meaning it is built on top of other containers like `deque` (by default), `vector`, or `list`. It provides a restricted interface for interacting with these underlying containers.

Important Modifiers of Containers Stack

- `empty()` : Time Complexity - O(1), constant time. It checks if the stack has no elements.
- `size()` : Time Complexity - O(1), constant time. It returns the number of elements in the stack.
- `top()` : Time Complexity - O(1), constant time. It returns a reference to the topmost element without removing it.
- `push(g)` : Time Complexity - O(1), constant time. It adds the element 'g' at the top of the stack.
- `pop()` : Time Complexity - O(1), constant time. It deletes the most recently entered element (the top element) from the stack.

Underlying Container of Stack

By default, the stack uses `deque` as the underlying container, but you can also specify other containers like `vector` or `list` if they support back insertion and deletion.

Customising the Stack's Underlying Container
You can create a stack with a custom underlying container like below:

```
stack<int, vector<int>> s;    // Stack using vector as the underlying
container
```

Applications of Stack

1. **Function Call Management**:
   Stacks are used to manage function calls in programming languages. When a function is called, its context (like local variables and return address) is pushed onto the stack, and when the function exits, the context is popped off the stack.
2. **Expression Evaluation**:
   Stacks are used to evaluate expressions in postfix (Reverse Polish Notation) and infix notation. They help in converting infix expressions to postfix and evaluating them efficiently.
3. **Backtracking Algorithms**:
   Stacks are used in backtracking algorithms, such as solving puzzles (like Sudoku) or navigating mazes. The stack keeps track of the choices made and allows the algorithm to backtrack to previous states.
4. **Undo Mechanisms**:
   Many applications, such as text editors and graphic design software, use stacks to implement undo functionalities. Each operation is pushed onto a stack, and popping an operation off the stack undoes the last action.
5. **Syntax Parsing**:
   Compilers use stacks to parse syntax and check for balanced parentheses or nested structures in code. The stack helps keep track of opening and closing brackets.
6. **Memory Management**:
   Stacks are utilized in managing memory allocation for local variables in programming languages, allowing for efficient use of memory.

7. **Depth-First Search (DFS)**:
   In graph traversal, stacks are used in the depth-first search algorithm to keep track of the nodes to be explored next.
8. **Web Browser Navigation**:
   Web browsers use stacks to maintain the history of visited web pages. The back button pops the current page off the stack and displays the previous one.
9. **Evaluation of Recursion**:
   When a recursive function is called, each call is pushed onto the call stack. This allows the program to keep track of the active function calls and return to the correct state after each call completes.
10. **Postfix Conversion and Evaluation**:
    Stacks are used to convert infix expressions (like `A + B`) to postfix expressions (like `AB+`) and evaluate them.

- Queue

Queue

Queue is a container adapter that provides a first-in, first-out (FIFO) data structure for storing elements. It is part of the sequence container adapters and is commonly used for scenarios where elements are processed in the order they are added, similar to a real-world queue.

Real-life Example

Consider a scenario where customers are waiting in line at a bank. A queue would be an appropriate choice for managing the order in which customers are served because:

- *First-In, First-Out (FIFO)*: The first customer to arrive is the first one to be served, and the order is maintained as additional customers join the queue.
- *Sequential Processing*: A queue naturally reflects the sequential nature of serving customers in the order they arrived.

Example Code

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    // Declaration and Initialization
    // Time Complexity: O(n) for initialization (where n is the number
of elements in the initializer list)
    queue<int> customerQueue({1, 2, 3, 4, 5});

    // Adding a customer to the queue
    // Time Complexity: O(1) for push
    customerQueue.push(6);

    // Serving customers
    cout << "Customers being served: ";

    // Looping through the queue
    while (!customerQueue.empty()) {
        // Accessing the front element
        // Time Complexity: O(1) for front
```

```
        cout << customerQueue.front() << " ";

        // Removing the front element
        // Time Complexity: O(1) for pop
        customerQueue.pop();
    }
    cout << endl;

    return 0;
}
```

Key Characteristics of Queue

- **FIFO (First In, First Out)**: The element that is added first will be the first one to be removed.
- **Container Adapter**: `queue` is a container adapter built on top of other underlying containers like `deque` (by default) or `list`, but it provides a restricted interface for interacting with these containers.

Important Modifiers of Containers Queue

- `empty()` : Returns whether the queue is empty. It returns true if the queue is empty otherwise returns false.
- `size()` : Returns the size of the queue.
- `swap()` : Exchange the contents of two queues but the queues must be of the same data type, although sizes may differ.
- `emplace()` : Insert a new element into the queue container, the new element is added to the end of the queue.
- `front()` : Returns a reference to the first element of the queue.
- `back()` : Returns a reference to the last element of the queue.
- `push(g)` : Adds the element ‘g’ at the end of the queue.
- `pop()` : Deletes the first element of the queue.

Underlying Container of Queue

By default, the `queue` uses `deque` as its underlying container because `deque` supports fast insertion and removal at both ends. However, you can specify a different underlying container, such as `list`, as long as it supports the necessary operations (insertion at the back and removal from the front).

Customising the Queue's Underlying Container
You can create a queue with a custom underlying container like below:

```
queue<int, list<int>> q;  // Queue using list as the underlying
container
```

Applications of Queue

1. **Job Scheduling**:
   Queues are widely used in job scheduling within operating systems. Processes waiting to be executed are placed in a queue, ensuring that jobs are executed in the order they arrive (First In, First Out - FIFO).

2. **Print Queue Management**:
   In printing tasks, documents sent to the printer are stored in a queue. This ensures that print jobs are processed in the order they are received.
3. **Breadth-First Search (BFS)**:
   Queues are essential in graph traversal algorithms like BFS. They keep track of nodes to be explored, ensuring that nodes are processed level by level.
4. **Customer Service Systems**:
   Queues are used in customer service scenarios (like call centers or bank lines), where customers are served in the order they arrive.
5. **Data Buffering**:
   Queues are utilised in data buffering, such as in IO Buffers or streaming applications, where data is processed in the order it is received.

- Priority Queue

Priority Queue

priority_queue is an implementation of a priority queue, which is a data structure that stores elements in a way such that the element with the highest priority is served before other elements with lower priority.
It is implemented as a heap and is part of the queue container adapters in the STL.

Real-life Example

Consider a scenario where tasks are assigned to a group of workers, and each task has a priority associated with it. A priority_queue would be a suitable choice for managing these tasks because:

- *Priority Handling*: The priority queue ensures that the task with the highest priority is processed first, simulating the real-world scenario where higher-priority tasks should be addressed before lower-priority ones.
- *Efficient Access*: The highest-priority task can be accessed efficiently, making it suitable for scenarios where quick access to the most critical element is crucial.

Example Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

int main() {
    // Declaration and Initialization
    // Time Complexity: O(n log n) for initialization (where n is the
number of elements)
    vector<int> initList = {3, 1, 4, 1, 5, 9, 2, 6};
    priority_queue<int> taskPriorityQueue(initList.begin(),
initList.end());

    // Adding a task
    // Time Complexity: O(log n) for push (assuming a binary heap)
    taskPriorityQueue.push(7);

    // Accessing the highest-priority task
    // Time Complexity: O(1) for top (since the highest-priority element
```

```
is always at the top)
    int highestPriority = taskPriorityQueue.top();
    cout << "Highest Priority Task: " << highestPriority << endl;

    // Processing the highest-priority task
    // Time Complexity: O(log n) for pop (assuming a binary heap)
    taskPriorityQueue.pop();

    // Displaying the remaining tasks
    cout << "Remaining Tasks: ";

    // Looping through the priority queue
    while (!taskPriorityQueue.empty())
    {
        // Accessing the highest-priority element
        // Time Complexity: O(1) for top (since the highest-priority
element is always at the top)
        cout << taskPriorityQueue.top() << " ";

        // Removing the highest-priority element
        // Time Complexity: O(log n) for pop (assuming a binary heap)
        taskPriorityQueue.pop();
    }

    cout << endl;

    return 0;
}
```

Key Characteristics of Priority Queue

- **Element Access Based on Priority**: In a priority queue, elements are served based on their priority. By default, the largest element (according to the `<` operator) is considered to have the highest priority. This behavior can be modified using custom comparators.
- **Heap-Based Implementation**: Internally, the priority queue is implemented as a binary heap (max-heap by default), which ensures efficient insertion and deletion of elements while maintaining the priority order.
- **Efficient Operations**: The priority queue supports efficient insertion (`push`), retrieval of the highest-priority element (`top`), and removal of the highest-priority element (`pop`) with logarithmic time complexity (`O(log n)`) except `top` .
- **Non-Sequential Access**: Unlike regular queues or vectors, a priority queue does not allow random or sequential access to its elements. It only allows access to the top element (highest priority) at any given time.
- **Comparator Customization**: You can provide a custom comparator (for example, to create a min-heap instead of a max-heap) when initializing a priority queue.

Important Modifiers of Containers Priority Queue

- `empty()` : Time Complexity - O(1), constant time. Indicates whether the priority queue is devoid of elements.
- `size()` : Time Complexity - O(1), constant time. Provides the count of elements present in the priority queue.

- `top()` : Time Complexity - O(1), constant time. Retrieves a reference to the highest-priority element in the queue.
- `push(g)` : Time Complexity - O(log n), where n is the number of elements in the priority queue. Appends the element 'g' to the end of the priority queue and adjusts the heap to maintain its structure.
- `pop()` : Time Complexity - O(log n), where n is the number of elements in the priority queue. Removes the foremost element from the priority queue and adjusts the heap.
- `emplace()` : Time Complexity - O(log n), where n is the number of elements in the priority queue. Inserts a new element into the priority queue, maintaining the order based on priority and adjusting the heap.

Underlying Container of Priority Queue

The priority queue in the C++ Standard Template Library (STL) uses one of the following containers as its underlying structure:

1. `vector` : By default, `priority_queue` uses `vector` as its underlying container, which provides efficient dynamic resizing and supports the heap operations required for the priority queue.
2. `deque` : While `vector` is the default container, the priority queue can also be implemented using `deque` (double-ended queue). However, this is not typical because `vector` provides better cache locality.
3. `less<T>` : By default, the comparator used for prioritising elements is `less<T>`, which creates a max-heap. You can customise this to use `greater<T>` to create a min-heap or any other custom comparator.

Applications of Priority Queue

1. **Task Scheduling**: Priority queues are widely used in operating systems for managing tasks with different priorities. The task with the highest priority is always processed first (e.g., job scheduling in CPU task management).
2. **Graph Algorithms**: Priority queues are essential in algorithms like Dijkstra's shortest path and Prim's Minimum Spanning Tree, where nodes with the lowest cost or priority are processed first.
3. **Event Simulation Systems**: In event-driven simulations, future events are processed based on their time of occurrence. Priority queues help in efficiently scheduling and executing events in the correct order.
4. **Real-Time Processing**: In systems requiring real-time data processing (e.g., network routers or firewalls), priority queues help prioritise packets or requests, ensuring timely responses.
5. **Huffman Coding**: The priority queue is crucial in Huffman coding (a compression algorithm) where characters are assigned variable-length codes based on their frequencies. The least frequent characters have higher priorities during the tree construction phase.

- Map

Map

Map is an associative container that stores elements in a key-value pair format.
It allows unique keys and fast retrieval of elements based on those keys.
It is implemented as a balanced binary search tree and is part of the associative container category.

Real-life Example

Consider a scenario where you need to store the ages of individuals by their names. A map would be a great choice for this task because:

- *Key-Value Association*: The map allows you to associate each person's name with their age efficiently.
- *Fast Look-up*: You can quickly retrieve a person's age by using their name as a key.

Example Code

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    // Declaration and Initialization
    // Time Complexity: O(n log n) for initialization (assuming n
elements and balanced tree construction)
    map<string, int> ages = {{"Alice", 25}, {"Bob", 30}, {"Charlie",
22}};

    // Inserting an element
    // Time Complexity: O(log n) for insert (assuming balanced tree)
    ages["David"] = 28;

    // Accessing elements
    // Time Complexity: O(log n) for accessing elements by key (assuming
balanced tree)
    cout << "Bob's age: " << ages["Bob"] << endl;

    // Iterating through elements
    // Time Complexity: O(n) for iterating through the map (where n is
the number of elements)
    for(auto it = ages.begin(); it != ages.end(); ++it) {
        cout << it->first << ": " << it->second << endl;
        cout << *it.first << ": " << *it.second << endl;    // Alternate
of above line
    }
        // Alternate way of iterating in map
        for(auto pair:ages) {
                cout << pair.first << ": " << pair.second << endl;
        }
        // Alternate way of iterating in map (By De-structuring)
        for(auto [f, s]:ages) {
                cout << f << ": " << s << endl;
        }

    // Finding elements
    // Time Complexity: O(log n) for find (assuming balanced tree)
    auto itFind = ages.find("Charlie");
    if (itFind != ages.end()) {
        cout << "Charlie's age is " << itFind->second << endl;
    }
    // Alternate way -> using count
    if(ages.count("Charlie")) {
```

```
            cout << "Charlie's age is " << itFind->second << endl;
    }

        // Remove single entry from map
        ages.erase("Bob");

        // Exchanges contents of 2 maps
        map<string, int> otherMap = {{"Kartik", 21}, {"Deepali", 20}};
        ages.swap(otherMap);

        // Remove all elements from map
        ages.clear();

    return 0;
}
```

Key Characteristics of Map

- **Associative Nature**: Maps are associative containers that store data in the form of key-value pairs, allowing efficient access and modification based on keys.
- **Unique Keys**: Each key in a map must be unique. If a new value is inserted with an existing key, it will overwrite the old value.
- **Ordered Elements**: Elements in a map are automatically sorted by the key using the less-than operator. This guarantees that iterating over the map will yield elements in ascending order of keys.
- **Dynamic Size**: Maps can grow or shrink dynamically as elements are added or removed.
- **Automatic Memory Management**: The map container automatically manages memory, ensuring that it allocates and deallocates memory as needed.

Important Modifiers of Containers Map

- `insert()` : Adds a new key-value pair to the map. If the key already exists, the value is updated.
- `erase()` : Removes an element with a specified key from the map.
- `clear()` : Removes all elements from the map, leaving it empty.
- `swap()` : Exchanges the contents of two maps, including their keys and values.
- `emplace()` : Constructs an element in-place and inserts it into the map, which can be more efficient than using `insert()` .
- `count()` : Returns the number of elements with a specified key in the map. For a `map` , this will either return `1` (if the key exists) or `0` (if the key does not exist), since `map` only allows unique keys.

NOTE
If you run the command `mp.erase("key")` on a key that doesn't exist in the map (`map<string, int> mp`), the following will happen:

1. **No Effect on the Map**: The `erase` function will simply do nothing if the specified key does not exist in the map. The map remains unchanged.
2. **Return Value**: The `erase` function returns the number of elements removed. If the key was not found, it returns `0` . If the key exists and is removed, it returns `1` .

Underlying Container of Map

The underlying data structure of `map` is typically a **Red-Black Tree**, a type of self-balancing binary search tree. This structure allows the map to maintain order and ensures that all operations (insertion, deletion, and lookup) have logarithmic time complexity, $O(\log n)$.

Applications of Map

1. **Database Indexing**: Maps are commonly used to index databases where fast retrieval based on a unique key (like a user ID) is essential.
2. **Caching Mechanisms**: Maps can be used to implement caches where keys represent request identifiers and values represent cached data.
3. **Counting Frequencies**: Maps are effective for counting occurrences of elements, such as counting word frequencies in a text.
4. **Configuration Management**: Maps can store configuration settings where the key represents the setting name and the value represents its value.
5. **Graph Representation**: Maps are used to represent graphs, where keys represent nodes and values are lists of adjacent nodes or weights of edges.

- Set

Set

set is an ordered collection of unique elements where each element must be unique. It is part of the associative container category and is implemented as a binary search tree.

Real-life Example

Consider a scenario where you want to maintain a list of unique user IDs who have registered for a service. A set would be an excellent choice for this task because:

- **Uniqueness:** A set ensures that each user ID is unique. You won't accidentally store the same user ID more than once.
- **Ordering:** The set maintains the user IDs in sorted order, making it easy to search for a specific user ID efficiently.

Example Code

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    // Declaration and Initialization
    // Time Complexity: O(n log n) for initialization (assuming n
elements and balanced tree construction)
    set<int> mySet = {5, 2, 8, 1, 9};

    // Insert Elements
    // Time Complexity: O(log n) for each insert (assuming balanced
tree)
    mySet.insert(6);
    mySet.insert(3);

    // Access Elements
    // Note: Sets do not provide direct access by index, as they are
ordered collections.
```

```
    // Remove Elements
    // Time Complexity: O(log n) for erase (assuming balanced tree)
    mySet.erase(2);

    // Size
    // Time Complexity: O(1) for size
    int size = mySet.size();

    // Iteration
    // Time Complexity: O(n) for iterating through the set (where n is
the number of elements)
    cout << "Set Elements: ";
    for (int element : mySet)
    {
        cout << element << " ";
    }
    cout << endl;

    // Clear
    // Time Complexity: O(n) for clear (assuming elements need to be
destroyed)
    mySet.clear();

    // Display the final set
    cout << "Final Set Elements: ";
    for (int element : mySet)
    {
        cout << element << " ";
    }
    cout << endl;

    return 0;
}
```

Key Characteristics of Set

- **Uniqueness**: The set container only stores unique elements. Duplicate values are automatically ignored if an attempt is made to insert them.
- **Ordered Elements**: Elements in a set are stored in a specific order (typically sorted in ascending order), which makes searching for a specific element efficient.
- **Efficient Operations**: Most operations in a set, such as insertion, deletion, and search, have logarithmic time complexity (`O(log n)`) due to the underlying binary search tree implementation.
- **No Random Access**: Unlike containers such as vectors or arrays, a set does not provide direct access to its elements using an index. You can only iterate through the elements sequentially.

Important Modifiers of Containers Set

- `insert(value)` : Inserts an element into the set if it's not already present. Time complexity is $O(\log n)$.
- `erase(value)` : Removes the specified element from the set. If the element is not found, nothing happens. Time complexity is $O(\log n)$.
- `clear()` : Removes all elements from the set, making it empty. Time complexity is $O(n)$ as it destroys all elements.

- `find(value)` : Searches for an element and returns an iterator to it if found, or to the end if not found. Time complexity is $O(\log n)$.
- `size()` : Returns the number of elements currently in the set. Time complexity is $O(1)$.
- `empty()` : Checks whether the set is empty. Time Complexity is $O(1)$.

Underlying Container of Set

The underlying container of a set is typically implemented as a **Red-Black Tree**, a self-balancing binary search tree. This structure ensures that the tree remains balanced after insertions or deletions, maintaining logarithmic time complexity for operations such as insertion, deletion, and search. This ordered structure is what ensures that the elements in a set are always sorted.

Applications of Set

1. **Storing Unique Values**: A set is commonly used to store unique elements where duplicates are not allowed, such as user IDs, IP addresses, or any other collection of unique identifiers.
2. **Efficient Searching**: The ordered nature of a set makes it useful for efficiently searching for elements, especially in cases where quick lookups are required, such as in membership checks or blacklist management.
3. **Removing Duplicates**: If you have a list or sequence of elements with duplicates, inserting them into a set will automatically remove any duplicates due to the set's uniqueness property.
4. **Mathematical Set Operations**: Sets can be used to perform set operations like union, intersection, and difference. These operations are useful in algorithms related to graph theory, databases, and probability.
5. **Data Deduplication in Algorithms**: Sets are used in many algorithms to avoid processing the same element multiple times (e.g., in depth-first search or breadth-first search to avoid revisiting nodes).

- Multiset and Multimap

Multiset and Multimap

Multiset and Multimap are associative containers that are similar to set and map, respectively, but they allow duplicate keys.
This means that multiple elements in these containers can have the same key.

Real-life Example
Consider a scenario where you need to store the ages of individuals, but some individuals might have the same age. A multiset or multimap would be suitable choices for these tasks because:

- *Duplicate Elements*: A multiset allows storing multiple occurrences of the same element (e.g., multiple individuals with the same age, meaning you can store the same age multiple times).
- *Duplicate Keys*: A multimap allows storing multiple key-value pairs with the same key (e.g., Multiple individuals associated with the same age, meaning you can store the same keys (age) with different values).
- *Efficient Storage*: These containers efficiently manage elements with the same key, making them useful in situations where duplicates are expected.

Example Code

```
#include <iostream>
#include <set>
#include <map>
using namespace std;

int main() {
    // Example of multiset
    // Declaration and Initialization
    // Time Complexity: O(n log n) for initialization (where n is the
number of elements in the initializer list)
    multiset<int> agesSet = {25, 30, 22, 25, 30};

    // Inserting an element
    // Time Complexity: O(log n) for insert (assuming balanced tree)
    agesSet.insert(25);

    // Count occurrences
    // Time Complexity: O(log n) for count (assuming balanced tree)
    int count = agesSet.count(25);
    cout << "Occurrences of age 25: " << count << endl;

    // Displaying the multiset
    // Time Complexity: O(n) for iterating through the multiset (where n
is the number of elements)
    cout << "Ages Set: ";
    for (int age : agesSet) {
        cout << age << " ";
    }
    cout << endl;

    // Example of multimap
    // Declaration and Initialization
    // Time Complexity: O(n log n) for initialization (where n is the
number of elements in the initializer list)
    multimap<string, int> agesMap = {{"Alice", 25}, {"Bob", 30},
{"Alice", 22}, {"Bob", 25}};

    // Inserting a key-value pair
    // Time Complexity: O(log n) for insert (assuming balanced tree)
    agesMap.insert(make_pair("Alice", 28));

        // Removing single (1st) occurrence of "Alice" from map
        auto it = agesMap.find("Alice");
        if(it != agesMap.end()) agesMap.erase(it);

        // Removing all occurrences of "Alice" from map
        agesMap.erase("Alice")

    // Displaying the multimap
    // Time Complexity: O(n) for iterating through the multimap (where n
is the number of elements)
    cout << "Ages Map: ";
    for (auto it = agesMap.begin(); it != agesMap.end(); ++it)
    {
        cout << it->first << ": " << it->second << " | ";
    }
```

```
    cout << endl;

    return 0;
}
```

Key Characteristics of Multiset and Multimap

1. **Multiset**
    - **Duplicates Allowed**: A multiset allows multiple occurrences of the same element (e.g., multiple values can be the same).
    - **Sorted Order**: Elements in a multiset are stored in a sorted order according to the specified comparator (default is less-than).
    - **No Key-Value Pairs**: A multiset only stores values and does not associate keys with those values.
    - **Dynamic Size**: The size of a multiset can grow or shrink as elements are added or removed.
2. **Multimap**
    - **Duplicate Keys Allowed**: A multimap allows multiple key-value pairs with the same key, meaning multiple values can be associated with a single key.
    - **Ordered by Key**: Elements in a multimap are sorted by keys, allowing for efficient retrieval based on the key.
    - **Key-Value Association**: Each element consists of a key and a value, providing a direct association between them.
    - **Dynamic Size**: Similar to multisets, multimaps can dynamically adjust their size.

Important Modifiers of Containers Multiset and Multimap

1. **Multiset**
    - `insert()` : Adds an element to the multiset, allowing duplicate values.
    - `erase()` : Removes a single occurrence of an element from the multiset.
    - `clear()` : Removes all elements from the multiset.
    - `count()` : Returns the number of occurrences of a specific element in the multiset.
    - `find()` : Finds the first occurrence of an element in the multiset.
2. *Multimap*
    - `insert()` : Adds a new key-value pair to the multimap; duplicate keys are allowed.
    - `erase()` : Removes all occurrences of a specific key from the multimap.
    - `clear()` : Removes all key-value pairs from the multimap.
    - `equal_range()` : Returns a range of elements matching a specific key, allowing access to all values associated with that key.
    - `find()` : Finds the first occurrenc__of a specific key in the multimap.

Underlying Container of Multiset and Multimap

Multisets are typically implemented as **Red-Black Trees** (a type of balanced binary search tree), which ensures that insertion, deletion, and search operations have $O(\log n)$ time complexity.

Like multisets, **Multimaps** are also implemented using **Red-Black Trees**, providing the same efficient performance characteristics for inserting, deleting, and finding key-value pairs.

Applications of Multiset and Multimap

1. **Multiset Applications**
   - **Counting Frequencies**: Useful for counting occurrences of elements, such as word frequencies in a document.
   - **Data Analysis**: Used in statistical analysis to maintain a collection of values with their frequencies.
   - **Histogram Representation**: Multisets can efficiently represent histograms, where the values represent data points, and their counts represent frequency.
   - **Event Tracking**: Useful in applications that need to track events and their occurrences.
   - **Sorting with Duplicates**: Multisets can be used when you want to maintain a sorted list with duplicate values.
2. **Multimap Applications**
   - **Database Indexing**: Useful for indexing database entries where multiple values can be associated with the same key (like multiple phone numbers for a contact).
   - **Configuration Management**: Can store settings where keys may have multiple values (like environment variables).
   - **Event Handling Systems**: Can map events to multiple handlers, allowing one event to trigger multiple actions.
   - **Graph Representations**: Multimaps can represent graphs where keys are nodes, and values are edges connecting those nodes.
   - **Key-Value Store**: Useful for applications where multiple values need to be associated with a single key for quick retrieval.

- Deque

Deque

A **deque** (double-ended queue) in C++ STL (Standard Template Library) is a sequence container that allows fast insertion and deletion of elements from both the front and back. It can be thought of as a hybrid of a vector and a list, combining the features of both while allowing dynamic array resizing.

Real-life Example

A *deque* can be used in scenarios where both front and back access to a collection of data is essential. For example, consider a *media playlist* in a music or video player. Users can add new tracks to the front (if they want it to play next) or to the back (to play later). They can also remove songs from both ends of the playlist, and the player can access tracks randomly based on user preference. The dynamic nature of the deque allows for efficient insertion and deletion at both ends, making it an ideal choice for such a task.

Example Code

```
#include <iostream>
#include <deque>
using namespace std;

int main() {
    deque<int> dq;
```

```
    // Inserting elements at the back and front
    dq.push_back(10);
    dq.push_back(20);
    dq.push_front(5);

    cout << "Deque after insertions: ";
    for (int x : dq) {
        cout << x << " ";
    }
    cout << endl;

    // Accessing elements
    cout << "Front element: " << dq.front() << endl; // Outputs 5
    cout << "Back element: " << dq.back() << endl;   // Outputs 20

    // Removing elements from the front and back
    dq.pop_front(); // Removes 5
    dq.pop_back();  // Removes 20

    cout << "Deque after removals: ";
    for (int x : dq) {
        cout << x << " ";
    }
    cout << endl;

    // Size of the deque
    cout << "Size of deque: " << dq.size() << endl;

    return 0;
}
```

Key Characteristics of Deque

- **Double-Ended**: You can add or remove elements from both ends (front and back) efficiently.
- **Dynamic Size**: Like vectors, deques can grow and shrink in size dynamically.
- **Random Access**: Deques allow random access to their elements, similar to arrays and vectors, but the access time is generally slower than vectors due to the way deques are implemented (as a series of contiguous memory blocks).
- **Non-Contiguous Memory**: Unlike vectors, the elements in a deque are not stored in a contiguous memory space, which allows for more flexibility in size but may have a performance trade-off.

Important Modifiers of Containers Deque

- `push_front(value)` : Adds an element to the front of the deque.
- `push_back(value)` : Adds an element to the back of the deque.
- `pop_front()` : Removes the element from the front of the deque.
- `pop_back()` : Removes the element from the back of the deque.
- `front()` : Returns a reference to the first element.
- `back()` : Returns a reference to the last element.
- `operator[]` : Provides access to elements using the subscript operator, allowing random access.

- `size()` : Returns the number of elements in the deque.
- `empty()` : Checks if the deque is empty.

Underlying Container of Deque

The underlying implementation of a deque in C++ STL is a dynamic array of pointers to blocks of contiguous memory. Unlike a vector, where all elements are stored in contiguous memory, a deque divides the elements into blocks. This structure allows the deque to efficiently handle dynamic memory allocation from both ends (front and back), making insertion and deletion operations at both ends constant time, i.e., $O(1)$.

Key points:

- *Non-contiguous memory blocks*: Elements in a deque are stored in separate memory blocks, allowing for flexibility in memory management.
- *Efficient resizing*: Unlike vectors, which require reallocation and copying when their capacity is exceeded, deques can grow and shrink by allocating or deallocating memory blocks at either end without requiring full reallocation.

Applications of Deque

1. **Buffer Implementation**:
   Deques are often used to implement buffers where data needs to be inserted and removed from both the front and the back. For example, a *sliding window* algorithm, often used in dynamic programming, can leverage a deque to track the current window of elements efficiently.
2. **Task Scheduling**:
   In real-time systems or task scheduling environments, deques can be used when tasks need to be added or processed from either the front or back of the queue. For example, higher-priority tasks may be added to the front, while lower-priority ones can be added to the back.
3. **Palindrome Checking**:
   Deques are useful for checking if a string or sequence is a palindrome, as you can compare characters from both ends. The *pop_front()* and *pop_back()* functions allow for efficient comparison from both ends of the sequence.
4. **Double-Ended Queues**:
   In operating systems, deques are used for scheduling processes or tasks where both ends of the queue need to be accessed for different purposes. For example, a *double-ended priority queue* can be used to handle both priority-based tasks and regular tasks efficiently.

NOTE :

Push() vs Emplace()

Push() vs Emplace()

`push()` and `emplace` are both used to add elements to containers like `queue`, but they differ in how they handle the insertion of elements.

1. **`push()`**
   - *Usage*: `queue.push(value);`
   - *How it works*: The `push` function creates a copy (or move) of the element and then inserts it into the container.
   - If you pass an object to `push`, it will call the copy constructor (or move constructor if applicable) to create the object before placing it into the container.

- Example:

```
queue<pair<int, int>> q;
q.push(make_pair(1, 2));  // Creates a pair, then copies it to the queue.
```

2. `emplace()`
   - *Usage*: `queue.emplace(args...);`
   - *How it works*: The `emplace` function constructs the element **in place** directly within the container without making a copy or move. It forwards the arguments to the constructor of the element and creates it directly in the container. This avoids an extra copy/move operation and can be more efficient, especially with complex objects.
   - Example:

```
queue<pair<int, int>> q;
q.emplace(1, 2);  // Constructs the pair directly in the queue.
```

Key Differences

- **Copy vs Direct Construction**
  - `push` requires creating an object first and then copying/moving it into the queue.
  - `emplace` constructs the object directly in place, which can be more efficient for complex objects.
- **Performance**
  - `emplace` can be faster than `push` since it eliminates the need for an extra copy or move.

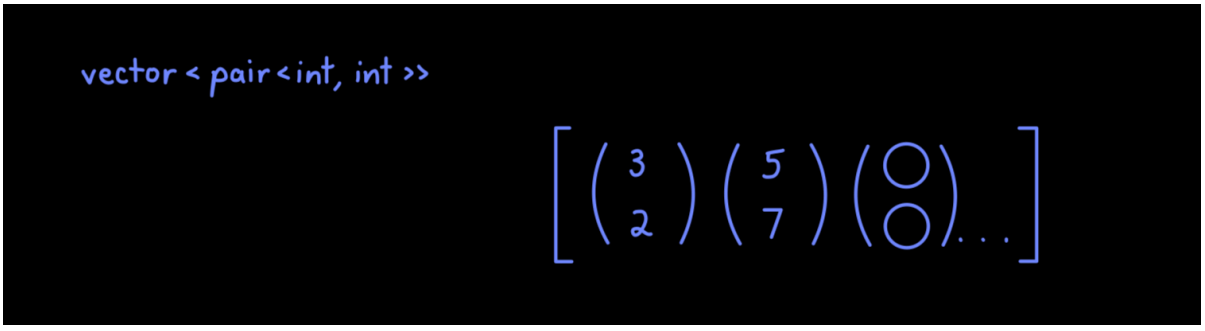
The thing that gives major power to STL is

Nesting

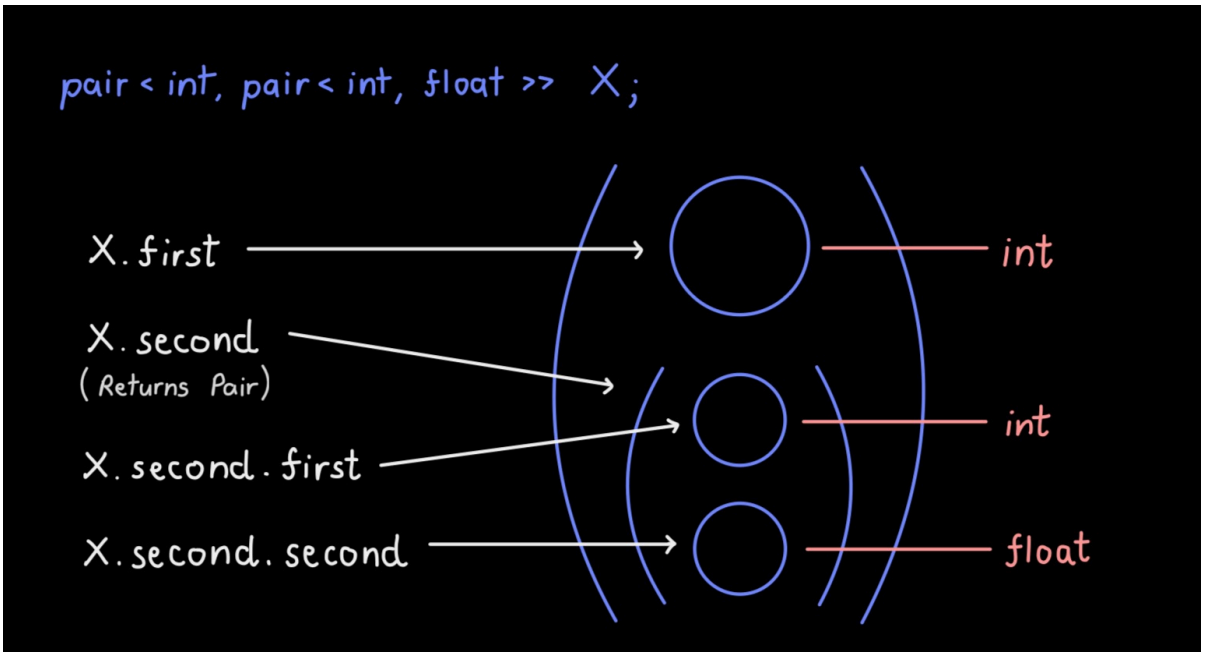
Nesting

Nesting in C++ STL (Standard Template Library) refers to the ability to use containers within other containers. This means you can create complex data structures by combining different STL containers. Nesting is particularly useful when you need to represent relationships between data or create multi-dimensional structures.

Example - 1 (Vector of Pairs)



Example - 2 (Pair of Pairs)



Key Points About Nesting in STL

- **Flexibility**: Nesting allows you to create flexible and complex data structures tailored to your needs.
- **Performance**: While nesting can simplify data management, it can also introduce overhead due to additional layers of indirection.
- **Readability**: Code readability may decrease with excessive nesting, so it's essential to strike a balance.
- **Container Types**: You can nest any type of STL container (e.g., vectors, sets, maps) within each other.

Problem Solving Paradigms (PSP)

Most of the problems you ever counter will belong to one amongst the following paradigms

1. *Brute Force* : Try all possible solutions and choose the best one, regardless of efficiency
2. *Greedy* : Make locally optimal choices at each step, hoping to find a global optimum
3. *Divide & Conquer* : Break the problem into smaller subproblems, solve each independently, and combine the results
4. *Dynamic Programming* : Recursion/Backtracking with Memory
5. *Graph* : Solve problems by modeling the relationships between elements as nodes and edges, often using search or traversal algorithms

Applications of STL

1. Budget Problem Variations

Budget Problem (Variation-1: Single Budget)

You're given an array A of size N, where `A[i]` denotes the cost of ith item. And you're also given a budget B, you've to tell how many maximum number of items could you buy with given budget.

Sort the array A and keep buying the smallest item until budget B is reached (Greedy)

```
int main() {
        int n;
        cin >> n;

        vector<int> arr(n);
        for(int i=0; i<n; i++) cin >> arr[i];
        sort(arr.begin(), arr.end());    // Sort items by prices

        int b;
        cin >> b;

        int cnt = 0;
        for(int i=0; i<n; i++) {
                if(arr[i] <= b) {
                        cnt++;
                        b -= arr[i];
                } else break;
        }
        cout << cnt << '\n';

        return 0;
}
```

Budget Problem (Variation-2: Budget Queries)

You're given an array A of size N, where `A[i]` denotes the cost of ith item. And you're also given M queries with budget B, and for each query you've to tell how many maximum number of items could you buy with given budget. ($N \leq 10^5$ and $M \leq 10^5$)

Approach-1

```
int main() {
        int n;
        cin >> n;

        vector<int> arr(n);
        for(int i=0; i<n; i++) cin >> arr[i];
        sort(arr.begin(), arr.end());    // Sort items by prices

        int m;
        cin >> m;

        while(m--) {
                int b;
                cin >> b;

                int cnt = 0;
                for(int i=0; i<n; i++) {
                        if(arr[i] <= b) {
                                cnt++;
                                b -= arr[i];
                        } else break;
                }
                cout << cnt << '\n';
```

```
        }

        return 0;
}
```

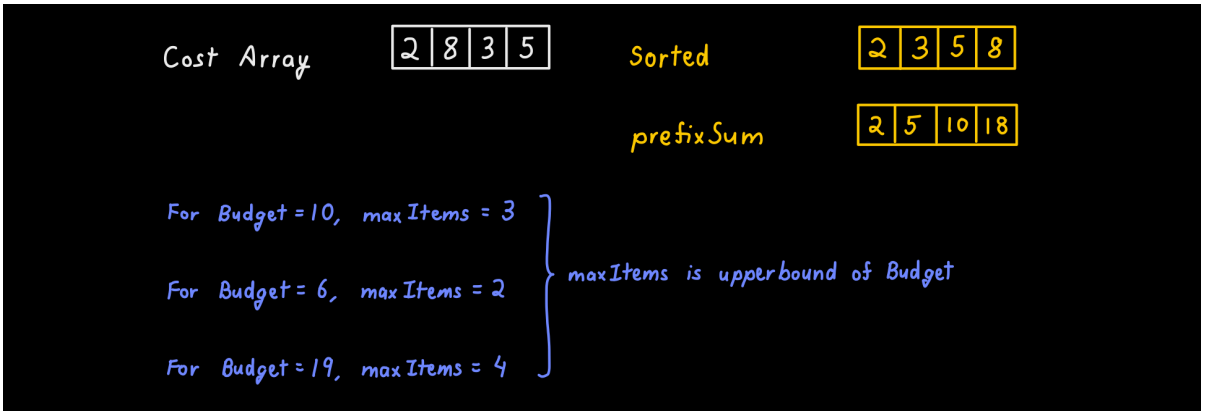
TC : $O(M \times N + N \log N)$
This TC comes out to be $O(10^{10})$ which will never pass for 1 sec time limit

---

Approach-2
After sorting, build prefixSum array
Since we want the maximum number of items we can buy with given budget, so it's simply the number of entries in prefixSum which are less than or equal to given budget

Since prefixSum is built after sorting the array, so prefixSum is itself sorted
Hence, we can apply upper bound to calculate number of entries less than or equal to budget

Dry Run



Code

```
int main() {
        int n;
        cin >> n;

        vector<int> arr(n);
        for(int i=0; i<n; i++) cin >> arr[i];
        sort(arr.begin(), arr.end());    // Sort items by prices

        // Build prefixSum
        vector<int> preSum(n);
        for(int i=0; i<n; i++) {
                preSum[i] = arr[i];
                if(i) preSum[i] += preSum[i-1];
        }

        int m;
        cin >> m;
        while(m--) {
                int b;
                cin >> b;
```

```
                int cnt = upper_bound(preSum.begin(), preSum.end(), b) -
preSum.begin();
                cout << cnt << '\n';
        }

        return 0;
}
```

TC : $O(N + M + N \log N)$
This TC comes out to be lesser than $O(10^8)$ which will always pass for 1 sec time limit

NOTE : For questions having queries, we generally prefer *Precomputation*

Related Topics : #Topic-Precompute

Related Concepts : prefixSum

2. Depth Counting

Depth Counting

Depth Counting in C++ generally refers to tracking the level of "nesting" or "depth" while traversing or parsing a structure, such as tree, graph, or string.

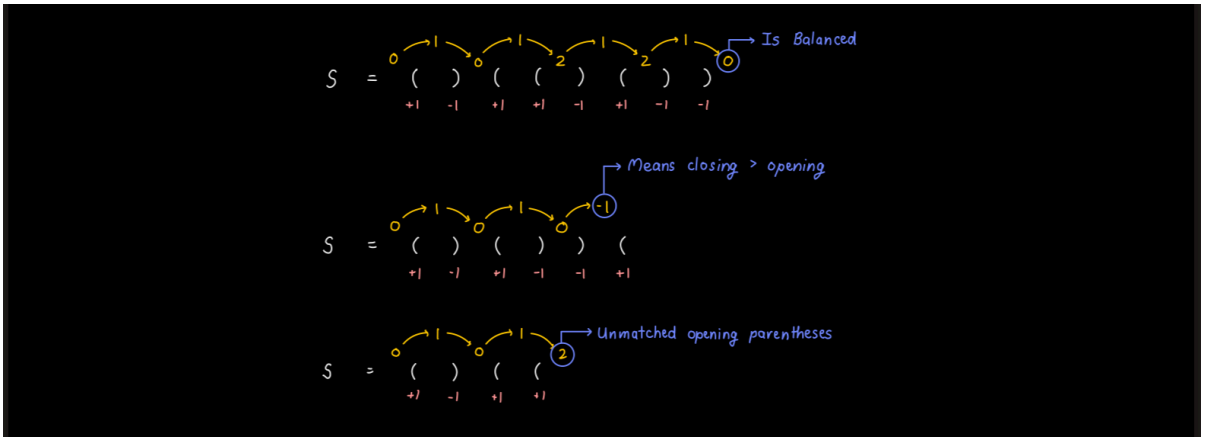
In case of Balanced Parentheses, depth counting helps us track the maximum depth of nested parentheses. This is particularly useful when we want to ensure that parentheses are balanced and properly nested in expressions.

Balanced Parentheses (Variation-1 : Single Type of Parentheses)

Balanced Parentheses means that :

- Every opening parenthesis `(` has a corresponding closing parenthesis `)`
- The parentheses are correctly nested, i.e., for every opening parenthesis `(`, the first unmatched closing parenthesis `)` must match it

Dry Run



Solution

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
        string s;
        cin >> s;

        bool isBalanced = 1;
        int depth = 0;    // Keeps track of how many unclosed opening parenthese are there
        for(auto ch:s) {
                if(ch == '(') {
                        depth++;
                } else {
                        depth--;
                }

                // If at any point depth becomes -ve
                // It means that no. of closing parentheses > no. of opening parentheses
                if(depth < 0) {
                        isBalanced = 0;
                        break;
                }
        }

        // If depth > 0, unclosed opening parentheses are remaining
        if(depth) isBalanced = 0;

        if(isBalanced) cout << "Balanced" << '\n';
        else cout << "Not Balanced" << '\n';

        return 0;
}
```

TC : $O(N)$
SC : $O(1)$

Balanced Parentheses (Variation-2 : Multiple Types of Parentheses)

Problem Link : [20. Valid Parentheses]

Platform : #Platform-LeetCode

Difficulty : #Difficulty-Easy

Can't be solved using 3 variables, for understanding dry run `{ [ ( ] ) } ( )` which isn't balanced

Closing of parenthesis should be done in exactly the reverse of order in which they appeared i.e. current Closing bracket should match the last unclosed Opening bracket

```
class Solution {
public:
        bool isValid(string s) {
                unordered_map<char, int> hash; // {parenthesis, mapped_number}
                hash['('] = 1;
                hash['['] = 2;
                hash['{'] = 3;
                hash[')'] = -1;
                hash[']'] = -2;
                hash['}'] = -3;

                stack<int> st; // Stores mapped_number
                bool isBalanced = 1;

                for(auto ch:s) {
                        if(hash[ch] > 0) {
                                st.push(hash[ch]);
                        } else {
                                // Parentheses matching logic
                                if(!st.empty() && hash[ch] + st.top() == 0) {
                                        st.pop();
                                } else return false;
                        }
                }

                // If stack isn't empty, that means some opening parentheses are still unclosed
                return (isBalanced && st.empty());
        }
};
```

TC : $O(N)$
SC : $O(N)$
Keeping map makes it easier to add new brackets without the need of manipulating the main code

Balanced Parentheses (Variation-3 : Query) [CHALLENGE PROBLEM]

You would be give a string `s` of size `N` containing `(` and `)`. You would be then given `q` queries in form of `? L R`, and you have to tell whether the substring from `L` to `R` is balanced or not.

Rephrasing the question to a bit easier version
Count the number of substrings that are balanced.

1st variation Solve it for $N \leq 10^3$
2nd variation Solve it for $N \leq 10^5$

HINT : Try drawing the depth values in form of mountain structures

For Query, L to R you might need some data structure, known as Segment Tree

Next Greater Element (NGE)

For every index you've to find the next index where the value is greater than the value at current index
NGE[i] $\rightarrow$ index of next greater element
Like NGE for `{5, 4, 2, 1, 3}` would be `{5, 5, 4, 4, 5}`
In case NGE of an element doesn't exist, then take size of array as NGE of that element

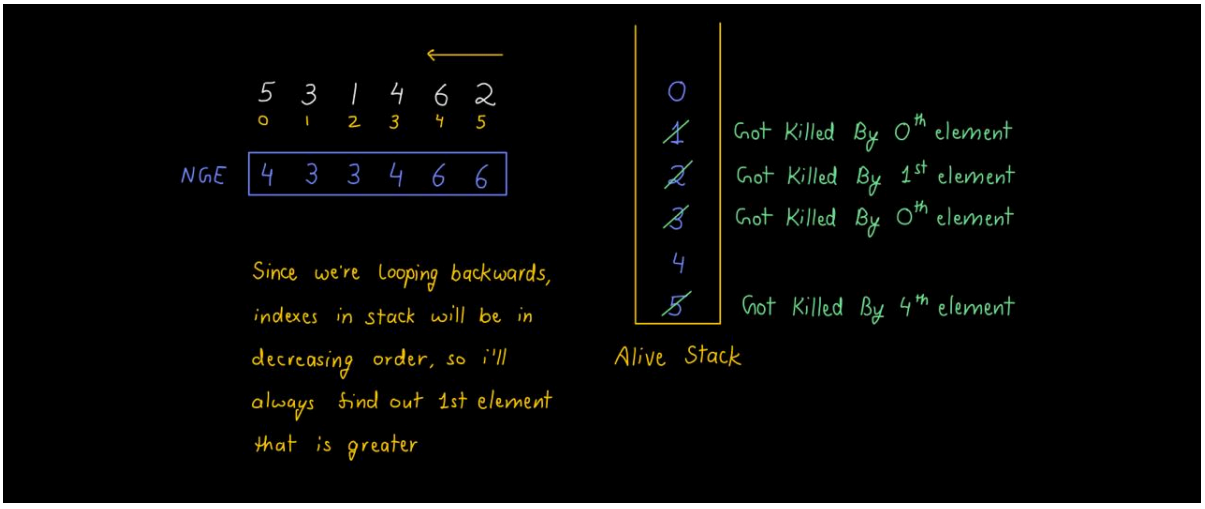
If a certain thing depends on things after it, then we process right to left (like `NGE[i]` always depends on elements present at right of `i`). Processing from right to left will help us build some more information by the time we reach a particular index & then we can use that information to calculate it faster, because $O(N)$ for whole array is like $O(1)$ per index.

Approach-1

Using Monotonic Stack (inserting in strictly decreasing order)

- Start traversing from right to left
- Kill all elements from stack which are smaller than currently encountered element
- If after all killings, if alive element(s) are left in stack, then it's the NGE, otherwise NGE doesn't exist
- Push the currently encountered element in the stack

Dry Run



Solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
        int n;
        cin >> n;

        int arr[n];
        for(int i=0; i<n; i++) cin >> arr[i];
```

```
        int nge[n];
        stack<int> alive;    // Stores indexes of elements that are alive
(haven't killed yet)

        for(int i=n-1; i>=0; i--) {
                // Maintain the decreasing order of stack by killing smaller
elements
                while(!alive.empty() && arr[alive.top()] <= arr[i]) {
                        alive.pop();    // Runs for N times
                }

                // If after all killings, something is alive, means it's my
next greater
                // Else next greater doesn't exists, so assign size of array
as next greater
                if(!alive.empty()) nge[i] = alive.top();
                else nge[i] = n;

                // Make the current element as alive
                alive.push(i);
        }

        // Print the nge
        for(int i=0; i<n; i++) cout << nge[i] << " ";

        return 0;
}
```

TC : $O(N)$ $\quad$ (Amortised Time Complexity)
SC : $O(N)$

Approach-2

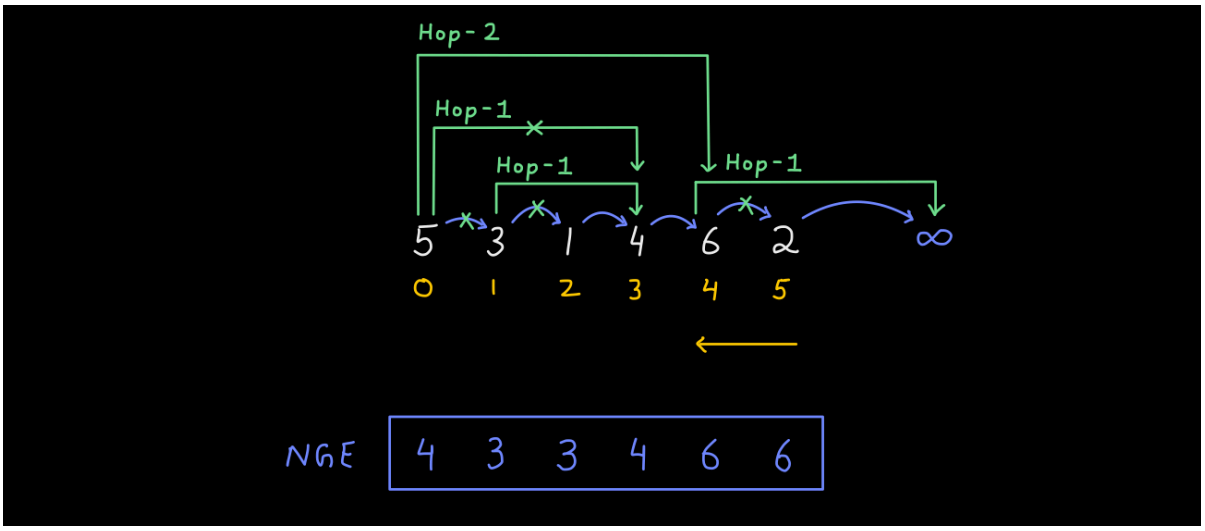
Without using any extra space

- Start traversing from right to left
- By default, take next index as nge i.e. `nge[i] = i+1`
- Till the nge exists & value we assigned to it is wrong, then assign next greater of nge as current nge i.e. `nge[i] = nge[nge[i]]`

If you're not greater than me, then i'll check with your's next greater element & so on
Because if you can't be my next greater, then any element between you and your next greater can't be my answer (because they'll are smaller than you), that's why i directly jump to your next greater element

The hopping going on here, is similar to killing of smaller elements like we were doing in Approach-1

Dry Run




```
#include <bits/stdc++.h>
using namespace std;

int main() {
        int n;
        cin >> n;

        int arr[n];
        for(int i=0; i<n; i++) cin >> arr[i];

        int nge[n];

        for(int i=n-1; i>=0; i--) {
                // By default, take nwext index as nge
                nge[i] = i+1;

                // Till the nge exists & it's assigned value is wrong,
                // Keep on assigning next greater of nge as current nge
(HOPPING)
                while(nge[i] != n && arr[i] <= arr[nge[i]]) {
                        nge[i] = nge[nge[i]];
                }
        }

        // Print the nge
        for(int i=0; i<n; i++) cout << nge[i] << " ";

        return 0;
}
```

TC : $O(N)$ $\quad$ (Amortised Time Complexity)
SC : $O(1)$

(Both challenge variations remaining)

3. Algorithmic Design

Algorithmic Design

Q1 Design a Data structure with following functions :

- insert(int x) : Inserts value `x`

- erase(int x) : Erases 1 occurrence of `x` (given `x` definitely exists at least once)
- erase_all(int x) : Erases all occurrences of `x`
- get_sum() : Returns sum of all entries present
- get_min() : Returns minimum of all entries present
- get_distinct() : Returns number of distinct entries amongst all entries present

Try using something called as *Frequency Map*
Keep in mind following points

- Map is sorted by Keys
- Entries in Map can be erased using Keys
- Accessing entry in Map is $O(\log N)$

```
#include <bits/stdc++.h>
using namespace std;

struct myBag {
        int curr_sum;
        map<int, int> freq;    // {entry, frequency}

        // Constructor
        myBag() {
                curr_sum = 0;
        }

        void insert(int x) {
                freq[x]++;
                curr_sum += x;
        }

        void erase(int x) {
                freq[x]--;
                // Delete entry when frequency becomes 0 (issue with get_min
& get_distinct)
                if(freq[x] == 0) {
                        freq.erase(x);
                }
                curr_sum -= x;
        }

        void erase_all(int x) {
                curr_sum -= freq[x] * x;    // Remove element from sum
frequency times
                freq.erase(x);
        }

        int get_sum() {
                return curr_sum;
        }

        int get_min() {
                return freq.begin()->first;    // (*freq.begin()).first
        }
```

```
        int get_distinct() {
                return freq.size();
        }
};

int main() {
        mybag x;
        x.insert(2);
        x.insert(5);
        cout << x.get_sum() << '\n';

        return 0;
}
```

Not using Unordered map because it's worst case accessing time complexity can go to $O(N)$, only average case is $O(1)$

NOTE : In 1 sec time limit, $O(N \log^2 N)$ may pass but $O(N \log^3 N)$ never passes

---

Q2 Design a Dashboard with following functions :

- ingest(name, score) : Inserts score (per ball) of Cricket Player with given name
- get_details(name) : Shows details (Sum, Mean, Variance and SD) of Player with given name

Will the data of 2 Players ever interact with each other ?
No, that's why use 2 structs

```
#include <bits/stdc++.h>
using namespace std;

struct Player_details {
        int cur_sum;
        int cur_cnt;    // Tracks number of balls played
        int cur_sq_sum;    // Tracks sum of squares of score

        // Constructor
        Player_Details() {
                cur_sum = 0;
                cur_cnt = 0;
                cur_sq_sum = 0;
        }

        void insert(int x) {
                cur_sum += x;
                cur_sq_sum += x*x;
                cur_cnt++;
        }

        // Converts cur_sum into double, to avoid integer division in
get_mean()
        double get_sum() {
```

```
                return cur_sum;
        }

        double get_mean() {
                return get_sum()/cur_cnt;    // double(cur_sum)/cur_cnt
        }

        // Var(X) = E(X^2) - E(X)^2
        double get_var() {
                double mean_sq_sum = (double)cur_sq_sum/cur_cnt;
                double mean = get_mean();
                return mean_sq_sum - (mean * mean);
        }

        // SD(X) = Var(X)^0.5
        double get_sd() {
                int var = get_var();
                return sqrt(var);
        }

        map<string, double> get_details() {
                map<string, double> details;
                details["mean"] = get_mean();
                details["sum"] = get_sum();
                details["variance"] = get_var();
                details["standard deviation"] = get_sd();
                return details;
        }
};

struct Dashboard {
        map<string, Player_details> mp;    // {name, details}

        void ingest(string player, int score) {
                map[player].insert(score);
        }

        map<string, double> get_details(string player) {
                return map[player].get_details();
        }
};

int main() {
        Dashboard mydash;
        mydash.ingest("virat",6);
        mydash.ingest("dhoni",2);
        mydash.ingest("virat",0);

        auto data = mydash.get_details("virat");
        for(auto v:data){
                cout << v.first << " " << v.second << '\n';
        }

        return 0;
}
```

NOTE : For learning about industry level good coding practices, refer C++ Style Guide

---

Q3 Design a Data structure with following functions :

- insert(int x) : Inserts value `x`
- erase(int x) : Erases value `x` (given `x` definitely exists at least once)
- get_mean() : Returns mean of all values present
- get_variance() : Returns variance of all values present
- get_median() : Returns median of all values present
- get_mode() : Returns mode of all values present (if frequency is same, consider highest valued one)

Handling the Median

The median is the value that separates the higher half from the lower half of a dataset. To efficiently maintain and compute the median during insertions and deletions, the `myBag` structure uses two `multiset`s:

- **`smaller`** : This multiset contains the smaller half of the numbers.
- **`larger`** : This multiset contains the larger half of the numbers.

$\underline{Insertion\ of\ an\ Element}$

When an element x is inserted into the `myBag`, the following steps are followed:

1. *Insert into the Appropriate Multiset*:
   - If the `smaller` set is empty, insert x directly into `smaller`.
   - If `smaller` is not empty, determine where x should go:
     - Compare x with the maximum of `smaller` (which is `*s_last`, where `s_last` is the last element of the `smaller` multiset).
     - If x is less than or equal to the maximum of `smaller`, it goes into `smaller`.
     - If x is greater, it goes into `larger`.
2. *Balance the Sets*:
   After inserting x, call the `balance_sl()` function to ensure the sizes of `smaller` and `larger` remain balanced:
   - The sizes can differ by at most 1.
   - If `smaller` has more than one extra element compared to `larger`, transfer the largest element from `smaller` to `larger`.
   - If `larger` has more elements than `smaller`, transfer the smallest element from `larger` to `smaller`.

$\underline{Removal\ of\ an\ Element}$

When an element x is erased, the process is as follows:

1. *Remove from the Appropriate Multiset*:
   Determine whether x is in `smaller` or `larger`:
   - If `larger` is empty, it implies x must be in `smaller`. Remove it from `smaller`.

- If x is less than or equal to the maximum of `smaller`, remove it from `smaller`.
- Otherwise, remove it from `larger`.

2. *Balance the Sets*:
   After removing x, call the `balance_sl()` function to ensure the sizes remain balanced.

Retrieving the Median

The median can be computed efficiently based on the sizes of `smaller` and `larger`:

- *If the total number of elements (`t_size`) is odd*:
  - The median is the maximum of `smaller`, which can be accessed as: cpp Copy code `return *(smaller.rbegin());`
- *If the total number of elements is even*:
  - The median is the average of the maximum of `smaller` and the minimum of `larger`. This can be computed as:

Handling the Mode

The mode is calculated by maintaining a frequency count of each element in the `freq` map. The `modeCalc` multiset is structured as pairs of frequency and value, allowing you to keep track of how many times each number appears.

Mode Handling Logic

- *Insert Operation*:
  - Before inserting a new element, if it already exists, we remove its previous frequency from `modeCalc` using `modeCalc.erase({freq[x], x});`.
  - We increment the frequency of the element in `freq`, and then insert the updated frequency into `modeCalc`.
  - This ensures that `modeCalc` always contains the most recent frequencies, allowing you to quickly retrieve the current mode.
- *Erase Operation*:
  - Similarly, when erasing an element, we first remove its frequency from `modeCalc`.
  - We decrement its frequency in `freq`, and if it's still greater than zero, we insert the updated frequency back into `modeCalc`.
  - If the frequency reaches zero, the element is removed from `freq`, ensuring that `modeCalc` only contains valid entries.

```
#include <bits/stdc++.h>
using namespace std;

struct myBag {
        multiset<int> smaller;
        multiset<int> larger;
        int s_size;
        int l_size;
        map<int, int> freq;     // {value: frequency}
        multiset<pair<int, int>> modeCalc;     // {frequency, value}
        int cur_sum;
        int cur_cnt;
        int cur_sq_sum;
```

```
myBag() {
        cur_sum = 0;
        cur_cnt = 0;
        cur_sq_sum = 0;
        s_size = 0;
        l_size = 0;
}

void balance_sl() {
        int d_size = s_size - l_size;
        if(d_size != 0 && d_size != 1) {
                if(d_size > 0) {
                        auto s_last = prev(smaller.end());
                        int transfer_val = *s_last;
                        smaller.erase(s_last);
                        s_size--;
                        larger.insert(transfer_val);
                        l_size++;
                }
                else {
                        auto l_first = larger.begin();
                        int transfer_val = *l_first;
                        larger.erase(l_first);
                        l_size--;
                        smaller.insert(transfer_val);
                        s_size++;
                }
        }
}

void insert(int x) {
        // Mean & Variance
        cur_sum += x;
        cur_sq_sum += x*x;
        cur_cnt++;

        // Mode
        if(freq[x] > 0) modeCalc.erase({freq[x], x});
        freq[x]++;
        modeCalc.insert({freq[x], x});

        // Median
        if(smaller.empty()) {
                smaller.insert(x);
                s_size++;
        } else {
                auto s_last = prev(smaller.end());
                if(x <= *s_last) {
                        smaller.insert(x);
                        s_size++;
                }
                else {
                        larger.insert(x);
                        l_size++;
                }
                balance_sl();
```

```
        }
    }

    void erase(int x) {
        // Mean & Variance
        cur_sum -= x;
        cur_sq_sum -= x*x;
        cur_cnt--;

        // Mode
        modeCalc.erase({freq[x], x});
        freq[x]--;
        if(freq[x] == 0) freq.erase(x);
        if(freq[x] > 0) modeCalc.insert({freq[x], x});

        // Median
        if(larger.empty()) {
            auto it = smaller.find(x);
            smaller.erase(it);
            s_size--;
        } else {
            auto s_last = prev(smaller.end());
            if(x <= *s_last) {
                auto it = smaller.find(x);
                smaller.erase(it);
                s_size--;
            } else {
                auto it = larger.find(x);
                larger.erase(it);
                l_size--;
            }
            balance_sl();
        }
    }

    double get_mean() {
        return double(cur_sum)/cur_cnt;
    }

    // Var(X) = E(X^2) - E(X)^2
    double get_variance() {
        double mean = get_mean();
        double mean_sq_sum = double(cur_sq_sum)/cur_cnt;
        return mean_sq_sum - (mean * mean);
    }

    double get_median() {
        int t_size = s_size + l_size;
        if(t_size % 2 != 0) return *(smaller.rbegin());
        else {
            double val_1 = *(smaller.rbegin());
            double val_2 = *(larger.begin());
            return (val_1 + val_2) / 2;
        }
    }

    double get_mode() {
```

```
                return modeCalc.rbegin()->second;
        }
};

int main() {
        myBag x;
        x.insert(2);
        x.insert(5);
        x.insert(8);

        cout << "Before erase" << '\n';
        cout << "Mean : " << x.get_mean() << '\n';
        cout << "Variance : " << x.get_variance() << '\n';
        cout << "Median : " << x.get_median() << '\n';
        cout << "Mode : " << x.get_mode() << '\n';

        x.erase(5);
        cout << '\n';

        cout << "After erase" << '\n';
        cout << "Mean : " << x.get_mean() << '\n';
        cout << "Variance : " << x.get_variance() << '\n';
        cout << "Median : " << x.get_median() << '\n';
        cout << "Mode : " << x.get_mode() << '\n';

        return 0;
}
```

Output

```
Before erase
Mean : 5
Variance : 6
Median : 5
Mode : 8

After erase
Mean : 5
Variance : 9
Median : 5
Mode : 8
```